SSW 590 — DevOps

DevOpsStartup: A Startup DevSecOps Assignment Manual

Version 1.43

Student Edition

by

Darian Muresan

Stevens Institute of Technology

October 28, 2026

© D. Darian Muresan (dmuresan@stevens.edu) 2026
ALL RIGHTS RESERVED

SSW 590 — DevOps

DevOpsStartup: A Startup DevSecOps Assignment Manual

Version 1.43
Student Edition

Darian Muresan
Stevens Institute of Technology
October 28, 2026

This release covers Gitea as the startup's source-control and issue-tracking hub, automated PDF builds of the team's LaTeX manual using Gitea Actions, Jenkins continuous integration with a required merge gate, and introductory SRE monitoring and reliability using Prometheus and Grafana. Each assignment has a 100-point rubric and requires an incremental update to the team's versioned DevOps IT manual. Further assignments can be added as the course develops.

Document History

Add the newest approved change at the top. Retain earlier entries and link each change to the affected chapter or controlled artifact.

Table 1: Assignment Manual Update History

Date / Version	Author	Change	Changed At
October 28, 2026 / 1.43	CDX; D. Muresan	Added Ansible secure delivery assignment, deployment diagram, two pricing releases, deployment starters, recovery exercises and rubric.	Chapter 6; delivery examples
October 21, 2026 / 1.42	CDX; D. Muresan	Simplified SRE scope to host metrics and HTTPS probes; made application metrics and full-day collection optional; aligned six panels, diagram, instructions, and rubric.	Chapter 5; Figure 5.1
October 14, 2026 / 1.41	CDX; D. Muresan	Required four published-image containers; added Compose installation and verification guidance; made SRE figure connectors orthogonal.	Chapter 5; Figure 5.1
October 14, 2026 / 1.40	CDX; D. Muresan	Widened the SRE deployment figure and separated labels, boxes, and connection routes.	Figure 5.1

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
October 14, 2026 / 1.39	CDX; D. Muresan	Activated introductory SRE assignment with Prometheus/Grafana deployment, implementation steps, SLO, alerts, incident exercise, PDF evidence, and aligned rubric.	Chapter 5; chapter map; references
October 7, 2026 / 1.38	CDX; D. Muresan	Increased Figure 4.2's vertical spacing and shortened its Jenkins node label to keep the citation clear.	Figure 4.2
October 7, 2026 / 1.37	CDX; D. Muresan	Clarified the optional build-agent container, controller-build alternative, checkout location, and matching diagram/PDF evidence.	Chapter 4; Figure 4.2; Rubric
October 7, 2026 / 1.36	CDX; D. Muresan	Added an optional complete Jenkinsfile for the Python example, with checkout, syntax, and test stages and execution-boundary notes.	Chapter 4; Appendix C; references
October 7, 2026 / 1.35	CDX; D. Muresan	Replaced reviewer-pending labels with D. Muresan in the AI assistance log and update history.	Appendix C; document history
October 7, 2026 / 1.34	CDX; D. Muresan	Added grouped September 30 and October 7 AI-assistance entries with build verification and instructor-review notes.	Appendix C
October 7, 2026 / 1.33	CDX; D. Muresan	Defined a local repository clone in a footnote beside the Chapter 4 pull-request example.	Chapter 4

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
October 7, 2026 / 1.32	CDX; D. Muresan	Added a sample issue-linked feature-branch push and Gitea pull-request workflow for the Jenkins merge-gate assignment.	Chapter 4; references
October 7, 2026 / 1.31	CDX; D. Muresan	Made IPv6 optional in Jenkins network checks and PDF evidence; retained IPv4 firewall verification in the rubric.	Chapter 4; Rubric
October 7, 2026 / 1.30	CDX; D. Muresan	Required a complete, clickable Jenkins HTTPS login-page URL and access instructions; clarified that a screenshot is supporting evidence.	Chapter 4; Rubric
October 7, 2026 / 1.29	CDX; D. Muresan	Added Jenkins HTTPS login-page availability evidence without requiring a grader account; aligned Step 2 and its rubric.	Chapter 4; Rubric
October 7, 2026 / 1.28	CDX; D. Muresan	Clarified that the written IT manual is primary, with labeled screenshots and links as evidence; aligned the Chapter 4 rubric.	Chapter 4; Rubric
October 7, 2026 / 1.27	CDX; D. Muresan	Shifted the deployment components and labels inside Figure 4.2's Droplet boundary slightly right.	Chapter 4; Figure 4.2
October 7, 2026 / 1.26	CDX; D. Muresan	Shifted the Cloud firewall right and tightened the Droplet boundary to make the browser-to-firewall arrow visible.	Chapter 4; Figure 4.2

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
October 7, 2026 / 1.25	CDX; D. Muresan	Clarified that teams may use the Python example as their tested application if they satisfy the same Gitea, Jenkins, merge-gate, and PDF evidence requirements.	Chapter 4
October 7, 2026 / 1.24	CDX; D. Muresan	Added minted syntax highlighting to the optional Python and shell examples and enabled shell escape in the assignment manual build.	Chapter 4; build configuration
October 7, 2026 / 1.23	CDX; D. Muresan	Added an optional Python example under the Jenkinsfile step, including syntax and unit-test commands and a controlled failed-check/repair demonstration.	Chapter 4
October 7, 2026 / 1.22	CDX; D. Muresan	Added a Chapter 4 footnote explaining how students create, commit, verify, and document the Jenkinsfile.	Chapter 4
October 7, 2026 / 1.21	CDX; D. Muresan	Increased and evened activity-block spacing so arrows remain visible; adjusted figure scale to fit the page.	Chapter 4; Figure 4.1
October 7, 2026 / 1.20	CDX; D. Muresan	Reworked the Jenkins–Gitea activity figure to separate steps and decisions and clarify the new-commit loop.	Chapter 4; Figure 4.1

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
October 7, 2026 / 1.19	CDX; D. Muresan	Added a Jenkins–Gitea activity diagram showing webhook, test, status, failed-check repair loop, peer review, and merge gate.	Chapter 4; Figure 4.1
September 30, 2026 / 1.18	CDX; D. Muresan	Removed the unrelated version-control product comparison from the Gitea Actions assignment.	Chapter 3
September 30, 2026 / 1.17	CDX; D. Muresan	Explained HTTPS Git submission through the TLS proxy, internal Gitea and optional SSH ports, and the separate Actions artifact and course-PDF paths.	Chapter 3; references
September 30, 2026 / 1.16	CDX; D. Muresan	Changed copyright attribution on the copyright page and page footers to D. Darian Muresan with email address.	Front matter; page footers
September 30, 2026 / 1.15	CDX; D. Muresan	Clarified where the LaTeX toolchain runs, how Gitea Actions reaches a temporary build container, and the host-mode alternative; added a deployment diagram and matching PDF evidence requirements.	Chapter 3; Figure 3.2; references
September 30, 2026 / 1.14	CDX; D. Muresan	Added a chapter-specific block diagram of the Gitea Actions manual-build and PDF-artifact flow.	Chapter 3; Figure 3.1

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
September 30, 2026 / 1.13	CDX; D. Muresan	Added Gitea Actions LaTeX PDF build assignment before Jenkins, with commit/run versioning, artifact evidence, and a 100-point rubric.	Chapter 3; Chapter 1; references
September 24, 2026 / 1.12	CDX; D. Muresan	Added an optional Caddy HTTPS implementation route and official setup links without supplying a finished configuration.	Chapter 2; references
September 24, 2026 / 1.11	CDX; D. Muresan	Applied the approved `dos` namespace; aligned chapter-specific TikZ sources and included AI guidance; added a Gitea deployment reference.	Chapter 2; Chapter 4; Appendices B–C
September 23, 2026 / 1.10	DDM, CDX	Added a linked Jenkins–Gitea source citation inside the deployment diagram.	Chapter 4; TikZ figure source
September 23, 2026 / 1.9	DDM, CDX	Moved the Jenkins deployment TikZ figure into its chapter-specific source folder.	Chapter 4; TikZ figure source
September 23, 2026 / 1.8	DDM, CDX	Added a Jenkins/Gitea UML deployment diagram and required teams to adapt it in their IT manuals.	Chapter 4
September 23, 2026 / 1.7	DDM, CDX	Added the Jenkins/Gitea integration assignment, explicit PDF evidence, security guidance, and aligned 100-point rubric.	Chapter 1; Chapter 4
September 23, 2026 / 1.6	DDM, CDX	Changed the title page label to Student Edition.	Title page

Assignment Manual Update History (continued)

Date / Version	Author	Change	Changed At
September 23, 2026 / 1.5	DDM, CDX	Named Let's Encrypt in the Gitea goal and clarified that it is a reasonable free TLS option with renewal verification.	Chapter 2
September 23, 2026 / 1.4	DDM, CDX	Made the PDF-only submission and eight rubric-aligned report updates explicit; removed the template attribution page.	Chapter 1; Chapter 2
September 23, 2026 / 1.3	DDM, CDX	Limited this release to the Gitea assignment; expanded firewall and database guidance; raised optional backup/restore credit to 20 points.	Chapter 1; Chapter 2
September 23, 2026 / 1.2	DDM, CDX	Made Gitea backup and restore optional extra credit, with evidence of an isolated restore test.	Chapter 2
September 23, 2026 / 1.1	DDM, CDX	Clarified that teams submit only the compiled IT manual PDF, not its LaTeX source.	Chapter 1; future chapters retained in source
September 23, 2026 / 1.0	DDM, CDX	Created modular startup DevSecOps assignments, shared standards, student-manual update requirements, and 100-point rubrics.	Chapters 1–2; future chapters retained in source

Table of Contents

Document History iv

1 Introduction and Common Requirements 1
- 1.1 Purpose and Chapter Map . . . 1
- 1.2 Rules for Every Assignment . . . 1
- 1.3 Submission and Grading . . . 2

2 Startup Foundation with Gitea 3
- 2.1 Description and Goal . . . 3
- 2.2 Required Work . . . 3
- 2.3 Suggested HTTPS Implementation: Caddy . . . 4
- 2.4 Database: Choose, Configure, and Verify . . . 4
- 2.5 Firewall and Network Exposure . . . 5
- 2.6 Exactly What to Update in the IT Manual PDF . . . 6
- 2.7 Rubric (100 Points) . . . 8
- 2.8 Optional Extra Credit: Verified Backup and Restore (Up to 20 Points) . . . 8

3 Automated LaTeX Manual Builds with Gitea Actions 9
- 3.1 Description and Goal . . . 9
- 3.2 How the Team Submits Source to Gitea . . . 10
- 3.3 Where the LaTeX Compiler Runs . . . 10
- 3.4 Required Work . . . 12
- 3.5 Exactly What to Update in the IT Manual PDF . . . 13
- 3.6 Rubric (100 Points) . . . 14

4 Continuous Integration with Jenkins 15
- 4.1 Description and Goal . . . 15
- 4.2 Required Work . . . 15
- 4.3 Activity Diagram: Pull Request to Merge . . . 18
- 4.4 Network and Trust Boundary . . . 20
- 4.5 Exactly What to Update in the IT Manual PDF . . . 20
- 4.6 Rubric (100 Points) . . . 22

5 Introduction to SRE: Monitoring and Reliability **23**
5.1 Description and Goal . . . 23
5.1.1 How This Assignment Extends the Earlier Work . . . 24
5.2 Deployment and Tool Responsibilities . . . 25
5.3 Required Implementation Steps . . . 25
5.3.1 Step 1: Establish a Baseline and Plan the Change . . . 27
5.3.2 Step 2: Deploy Persistent, Private Monitoring . . . 27
5.3.3 Step 3: Verify Host Metrics and Collection Health . . . 31
5.3.4 Step 4: Measure HTTPS Availability with Probes . . . 32
5.3.5 Step 5: Build a Dashboard That Answers Operational Questions . . . 32
5.3.6 Step 6: Define and Evaluate One Availability Objective . . . 33
5.3.7 Step 7: Configure and Test Two Actionable Alerts . . . 34
5.3.8 Step 8: Run a Controlled Incident and Verify Recovery . . . 36
5.3.9 Common Problems and Checks . . . 36
5.4 What to Turn In: One Updated Team IT Manual PDF . . . 37
5.5 Rubric (100 Points) . . . 38

6 Secure Delivery and Rollback **40**
6.1 Description and Goal . . . 40
6.2 Starter Application and Expected Behavior . . . 40
6.3 Deployment Architecture . . . 41
6.4 Implementation Steps . . . 42
6.4.1 1. Prepare the Repository and Establish a Baseline . . . 42
6.4.2 2. Prepare the Trusted Agent and Target . . . 42
6.4.3 3. Connect the Application to the Existing Proxy . . . 44
6.4.4 4. Build Once and Record an Immutable Release . . . 45
6.4.5 5. Apply Proportionate Security Gates . . . 45
6.4.6 6. Deploy v1 with Ansible and Prove Idempotence . . . 46
6.4.7 7. Approve and Promote the Same Image . . . 46
6.4.8 8. Release v2 and Exercise Two Recovery Cases . . . 47
6.4.9 9. Reuse Monitoring and Document Operations . . . 47
6.5 What to Turn In . . . 48
6.6 Rubric (100 Points) . . . 48
6.7 Official References . . . 49

A Student IT Manual Update Guide **50**
A.1 Minimum Table Fields . . . 50
A.2 Evidence Standard . . . 50

B AI Agent Usage Guide **51**

C AI Assistance Log **52**

Bibliography **55**

Glossary **60**

Acronyms **61**

Index **62**

List of Tables

1 Assignment Manual Update History iv

List of Figures

2.1 Reference Gitea deployment on one Droplet. The Cloud Firewall admits HTTPS on 443 and, when needed, HTTP on 80 for redirect or certificate validation; it limits administrator SSH. The reverse proxy alone publishes web ports. Gitea's backend and the selected SQLite or PostgreSQL data path remain private (PostgreSQL uses internal port 5432). Adapt this reference to the actual deployment [28, 13]. 6

3.1 From a Gitea source push to a versioned, downloadable manual PDF. 10
3.2 Gitea Actions runner launching an isolated LaTeX build job on the same Droplet. 11

4.1 Jenkins–Gitea pull-request activity. A new commit repeats the build; the reported check on the latest head commit, not webhook delivery, gates merging. Sources: [29, 41]. 19
4.2 UML deployment reference for one-Droplet Gitea and Jenkins integration. The browser reaches the TLS proxy through firewall-approved HTTPS (443). Webhook, repository checkout, status, and data paths stay private. Builds in this reference run on a Jenkins controller executor. The Gitea data node represents the team's chosen database and persistent repository storage. . . 20

5.1 Existing Gitea/Jenkins deployment extended with monitoring. Each blue component is a separate container. All connectors use horizontal and vertical segments. Internal ports are examples, not public firewall allowances. Verify the probe's actual hostname/DNS route. Based on the official component documentation [57, 36, 56, 54]. 26

6.1 Application delivery on the existing Droplet. Arrows show the initiator of each labeled action; approval precedes production deployment. Blue identifies Chapter 6 additions, white identifies existing components, and gray identifies persistent storage or read-only host mounts. Dotted lines show mounts. Existing infrastructure and its data remain in service. 43

Chapter 1

Introduction and Common Requirements

1.1 Purpose and Chapter Map

Your team is operating a small startup, not assembling unrelated demos. Each assignment adds a working capability to the same product, repository, service, and operational manual. The assignments are modular: the instructor will announce the order and due dates, and may add, remove, or revise chapters. Each chapter states its own objective, deliverables, and 100-point rubric. This release contains the Gitea foundation, automated LaTeX manual build, Jenkins integration, and introductory SRE monitoring assignments; later chapters will be activated when they are ready.

Task	Goal
2	Establish the startup's Gitea work hub, student domain, HTTPS, and minimal public exposure.
3	Build the team's LaTeX manual on each Gitea push, embed its commit/run identity, and publish the PDF.
4	Connect Jenkins to Gitea so automated tests and a required status check protect reviewed merges.
5	Monitor Gitea, Jenkins, and the droplet with Prometheus/Grafana; measure an SLO and practice alerting and recovery.

1.2 Rules for Every Assignment

1. Maintain the team's *separate* LaTeX DevOps IT manual, beginning with the course `main/DevOpsTemplate/`. Submit only its compiled PDF on every assignment; retain the LaTeX source within the team. The manual is a living operational record, not a pasted collection of screenshots.

2. Put a version number on every submitted document. Add an update-history entry for

initial setup and each assignment change, with date, author, short summary, and links to affected chapters or evidence. Explicitly name any Hosts or Passwords table changes in that entry.

3. Keep Hosts and Passwords tables current. The Passwords table contains *hints only*: no passwords, derivation keys, recovery codes, tokens, or private keys. Use distinct secrets for distinct services and store real credentials outside the report and repository.

4. Use clear headings, labeled figures/tables, a glossary, an index, and citations where useful. Provide sufficient configuration and test evidence for another team to reproduce and verify the work. Redact sensitive values from screenshots and logs. Attribute reused code, images, and AI assistance.

5. All user-facing web interfaces must use valid hostname-based HTTPS. HTTP may be open only for redirection or certificate validation. Do not expose raw application, database, metrics, or Jenkins agent ports publicly. Restrict SSH to approved source IPs or a private access path. Document cloud firewall *and* host/container networking rules; Docker-published ports can bypass some host firewall expectations, so test exposure externally [13, 14]. Internal service-to-service traffic may use an isolated private network; document the trust boundary.

6. Use least privilege, pinned or recorded software versions, persistent data, and repeatable configuration. Never claim an unrun test passed. Describe failed attempts and corrective actions. Run disruptive experiments only on your own authorized resources, with a recovery path.

7. Record the domain offer, cloud services used, possible charges/renewals, and shutdown or cleanup steps. Student offers and prices can change; verify current terms rather than assuming they are free indefinitely.

1.3 Submission and Grading

Submit *one artifact only*: the updated, compiled PDF of the team's DevOps IT manual. Do not submit the LaTeX source, separate screenshots, configuration files, or backup archives. Put the required explanations, sanitized configuration excerpts, labeled evidence, and clickable links to the team's Gitea repository and HTTPS hostname *inside the PDF*. Keep the versioned non-secret deployment files in the team repository and link to them from the PDF. The instructor's learning-management system sets the due date and submission location. Each chapter's rubric totals 100 points. Missing evidence receives no credit for the claim it was meant to support. Security requirements are graded in the relevant rubric row; a secret exposed in a submission must be rotated and reported promptly. A missing or stale Hosts or Passwords table incurs a 10-point deduction from the assignment score (minimum zero); do not deduct for the same omission twice.

Chapter 2

Startup Foundation with Gitea

2.1 Description and Goal

Create the startup's first operational work hub on DigitalOcean. Use an existing, version-pinned Gitea container image; building an image from source is *not* required. A real startup team should be able to file an issue, review a proposed change, and find its history in Gitea. This assignment establishes the domain, HTTPS, access controls, and persistent data on which later tasks depend [28]. Let's Encrypt provides free, automated TLS certificates and is a reasonable choice for this student deployment; the team must still configure renewal and verify the live HTTPS service [51].

2.2 Required Work

1. Claim an eligible domain offer through the GitHub Student Developer Pack. Verify eligibility, first-year cost, renewal cost, and expiration [34]. Point a hostname such as `git.example.com` at the team's DigitalOcean Droplet; show the DNS record and an independent lookup.

2. Deploy Gitea using a pinned official image and versioned Compose file. Choose and document SQLite or PostgreSQL as described below. Explain the volumes, restart behavior, image source, and non-secret configuration. Prove repositories and issues survive a container replacement.

3. Put a reverse proxy or equivalent TLS endpoint before Gitea. Research two certificate options and justify one; Let's Encrypt is a reasonable option. Show a valid certificate for the hostname, HTTP-to-HTTPS redirect, expiration date, automated renewal configuration, and a successful renewal test or dry run. State where TLS terminates and how private traffic reaches Gitea. Certificate challenge choices have different port/DNS requirements [50].

4. Configure the DigitalOcean firewall and host/container publishing using the firewall requirements below. Verify the permitted and blocked ports from outside the Droplet. Keep database and application-backend ports private [13, 14].

5. Create a startup organization, product repository, README, and at least two member accounts with appropriate roles. Create at least five actionable issues, including product and operations work. Demonstrate two issue-linked branches/pull requests, peer review, merge, and issue closure.

2.3 Suggested HTTPS Implementation: Caddy

Caddy is one approachable way to satisfy the reverse-proxy requirement; it is a suggestion, not a required product. Its official Docker Compose guide shows how to run the proxy container and persist certificate data [9]. The Caddy reverse-proxy guide explains how to route a public hostname to an application, while its automatic HTTPS guide explains certificate issuance, renewal, and HTTP-to-HTTPS redirects [10, 8].

If your team chooses Caddy, use those guides to develop its own Compose and proxy configuration. Point the chosen DNS hostname at the Droplet; place Caddy and Gitea on a shared private Compose network; publish only the proxy's web ports; and route requests to Gitea by its Compose service name, not `localhost`. Persist Caddy's certificate state and set Gitea's public `ROOT_URL` to the HTTPS hostname [20]. Decide whether port 80 is needed for certificate validation and redirect, and record that decision in the firewall rules. Then verify the live certificate, redirect, renewal mechanism, and that Gitea's backend port is not publicly reachable. These are implementation goals, not a supplied configuration; the team's PDF must explain and prove its own choices.

2.4 Database: Choose, Configure, and Verify

Gitea stores issues, users, organizations, pull-request metadata, and settings in a database; Git repositories and some attachments also live in persistent application data. Both parts must survive replacement of the Gitea container. Choose *one* of the following paths and identify it in the architecture diagram [28, 22]. Figure 2.1 shows both supported alternatives; a team deploys only its selected path.

SQLite (simpler)
: Use Gitea's built-in SQLite database for a small student team. Record the database-file location inside the persistent Gitea volume (normally under `/data`), and show that the volume is mounted outside the disposable container layer. No database service or public database port is needed. Explain that SQLite is a reasonable small-team choice but may not fit a larger, higher-concurrency deployment.

PostgreSQL
: Run a version-pinned PostgreSQL container with its own persistent volume. Connect Gitea to it by Compose service name and internal port 5432 on a private Compose network; do not publish 5432 to the Droplet. Record database name, application account, image tag, volume, network, and connection settings. Put the database credential in an untracked, restricted secret file or

equivalent secret store; show a redacted configuration and explain who can read the secret. Do not include credentials in the PDF, screenshots, Compose file, or Git repository.

For either path, create a repository and an issue, record their identifiers, replace only the Gitea container without deleting volumes, and verify both remain available. Record the exact command, before/after evidence, and a short explanation of why the data persisted. Show how the database is connected to Gitea and why it is not reachable from the public internet. The persistence check is *not* a backup or restore test. Teams using PostgreSQL should also show the database container is healthy after restart and explain how Gitea behaves if that service is unavailable.

2.5 Firewall and Network Exposure

HTTPS protects traffic to the web interface; it does not close other ports. Use the DigitalOcean Cloud Firewall as the public network boundary, then check the Droplet and Docker port mappings. Apply the firewall to the correct Droplet and document its name, rules, source ranges, and date. DigitalOcean firewalls are stateful and admit only traffic allowed by their rules [13, 12]. Do not rely on host `ufw` alone: Docker-published container ports can bypass expected `ufw` filtering [14].

Service	**Public source**	**Required treatment**
443/TCP	All users	Allow to the HTTPS endpoint for Gitea.
80/TCP	All users, if used	Allow only for HTTP-to-HTTPS redirect or certificate validation; explain its purpose.
Admin SSH	Approved team IPs or private access path	Restrict port 22 (or the chosen admin port); establish and test this rule before removing broader access to avoid lockout.
Gitea backend	None	Connect proxy to Gitea privately (often port 3000); do not publish this port publicly.
PostgreSQL	None	Internal container network only (port 5432); no public host-port mapping. SQLite needs no network port.
Git over SSH (optional)	Documented users	Prefer Git over HTTPS for this assignment. If enabled, document a distinct port and the justified firewall rule; do not confuse it with admin SSH.

Document the inbound rule table for both IPv4 and IPv6 if the Droplet has both. Explain outbound rules at a high level: the Droplet must still reach needed package/image sources, DNS, time services, and certificate services. Do not blindly block all outbound traffic. Check for overlapping Cloud Firewalls whose rules might change the effective policy [13].

Provide a small network diagram showing browser to 443, optional 80 to the proxy, proxy to private Gitea backend, and Gitea to its private database or SQLite file. Figure 2.1 is a reference only: adapt it to the team's actual services, network, and database choice. Include

a sanitized Cloud Firewall screenshot or rule export, the Compose port mappings, and results of an *external* check from a machine or network outside the Droplet. Show that 443 works, 80 behaves as documented, admin SSH is restricted, and the backend/database ports cannot be reached publicly. A scan run on the Droplet itself does not prove public exposure. Record the test source, time, expected result, actual result, and any correction. Test only your own assigned Droplet.

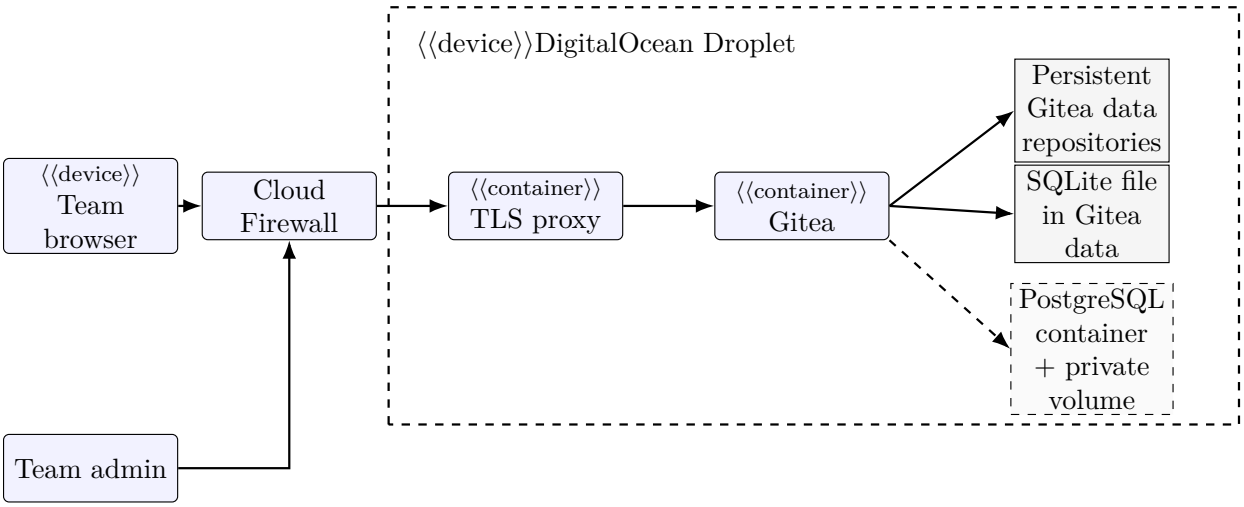


Figure 2.1: Reference Gitea deployment on one Droplet. The Cloud Firewall admits HTTPS on 443 and, when needed, HTTP on 80 for redirect or certificate validation; it limits administrator SSH. The reverse proxy alone publishes web ports. Gitea's backend and the selected SQLite or PostgreSQL data path remain private (PostgreSQL uses internal port 5432). Adapt this reference to the actual deployment [28, 13].

2.6 Exactly What to Update in the IT Manual PDF

Update the team's existing manual, then submit *only its compiled PDF*. Add a *Gitea* chapter and put the following eight numbered subsections in it. Each item matches one row of the 100-point rubric, in the same order. Embed labeled, dated, sanitized screenshots or command output where requested; put clickable links and redacted configuration excerpts in the PDF rather than submitting separate evidence files. A screenshot alone does not replace an explanation of the configuration or test result.

1. **Student Pack domain and DNS (10 points).** Identify the domain offer, eligibility, first-year and renewal costs, expiration, and the chosen Gitea hostname. Show the DNS record, independent lookup result, and HTTPS hostname link. Explain how a future team will track renewal.

2. **Pinned Gitea deployment (10 points).** Record Droplet and Gitea image versions, image source, deployment commands, restart policy, persistent mounts, and proxy connection. Include a redacted Compose excerpt and a link to the versioned non-secret

deployment files in the team repository. Explain how another team could repeat the installation.

3. **Database and persistence (10 points).** State whether the team uses SQLite or PostgreSQL and document the corresponding settings, private connection or file path, volumes, and secret handling described in Section 2.4. Give the repository/issue identifiers, exact container-replacement command, before/after evidence, and conclusion. For PostgreSQL, include the post-restart health check.

4. **HTTPS and certificate renewal (15 points).** Compare two certificate options, justify the selected one, and identify TLS termination. Show the valid hostname certificate, issuer and expiry, HTTP-to-HTTPS behavior, renewal automation, and successful renewal test or dry run. Include the relevant redacted proxy configuration and test output.

5. **Firewall and external exposure (15 points).** Include the DigitalOcean Cloud Firewall name and Droplet attachment, inbound rules and source ranges, IPv4/IPv6 treatment, host/Compose port mappings, and a small network diagram. Record external test source and time, expected versus actual results for 443, optional 80, admin SSH, Gitea backend, and database port, plus any corrections. Explain why HTTPS does not replace a firewall.

6. **Startup team workflow (20 points).** Link the organization and product repository; show the README, two members and their roles, and at least five actionable issues. Trace two issues through linked branches/pull requests, peer review, merge, and closure, using links or labeled evidence.

7. **Architecture and operation (5 points).** Explain the end-to-end request and data paths with an architecture diagram. State where data lives, which interfaces are public or private, and how the team restarts and troubleshoots the service. Distinguish the database-persistence test from a backup/restore test.

8. **Manual quality and controls (15 points).** Increase the manual's version; add a dated history entry for initial setup and this update with links to changed sections and evidence. Update the Hosts and Passwords tables (password *hints only*) and mention both in the history. Use clear headings, captions, citations, glossary/index entries where useful, and redact all secrets. State any failed tests and corrective actions.

The optional backup-and-restore evidence belongs in a separate, clearly labeled extra-credit subsection of the *same PDF*; it is not part of the eight required rubric items.

2.7 Rubric (100 Points)

Criterion	Points
Student Pack domain, cost/renewal check, DNS evidence	10
Pinned Gitea deployment and reproducible configuration	10
Database configuration and persistence verification	10
Valid HTTPS, redirect, certificate details, renewal test	15
Firewall, limited exposure, external verification	15
Startup organization, backlog, issue-to-reviewed-merge workflow	20
Architecture and operational explanation	5
Versioned IT manual, history, Hosts/Passwords hints, citations and evidence	15
Total	**100**

2.8 Optional Extra Credit: Verified Backup and Restore (Up to 20 Points)

Backup is *not* required for this assignment and earns no points in the 100-point rubric. For up to 20 extra-credit points, back up both Gitea's persistent application data (including repositories and attachments) and its database, then restore them to a separate, temporary Gitea instance. Do not overwrite the working startup service.

Document the backup method, what data it covers, where it is stored, and when it ran (5 points). Show a sanitized backup log or file listing and explain how the database and application data belong to the same recoverable state (5 points). Create a small test repository and issue before the backup, then show that both are present and usable in the restored instance (10 points). Include the restore log and labeled screenshots of the original and restored repository and issue, plus the test date and outcome. A backup file or successful backup command *alone* does not prove recovery. Do not submit backup archives, database dumps, credentials, or private keys.

Chapter 3

Automated LaTeX Manual Builds with Gitea Actions

3.1 Description and Goal

Turn the startup's DevOps IT manual into a reproducible build. Keep its LaTeX source in the Gitea repository established in Chapter 2. Each push to the chosen branch must trigger a *Gitea Actions* workflow that compiles the manual and makes the resulting PDF available as a downloadable build artifact. This assignment uses a self-managed Gitea runner, not a GitHub-hosted runner or Overleaf. Jenkins integration remains a separate assignment in Chapter 4 [24, 23].

The PDF must identify both the triggering Git commit and the workflow run. A push may contain several commits. Use the triggering commit's short hash as the source revision and the Gitea workflow run number as the build identifier. Keep the manual's human-maintained version (for example, `v1.3`) and append a build suffix such as `+build.42.a1b2c3d`. A re-run may retain the same run number; if distinguishing attempts matters, include the run-attempt number as well [25].

Figure 3.1 summarizes the expected path. It is a conceptual block diagram; teams choose and document their own runner and LaTeX toolchain.

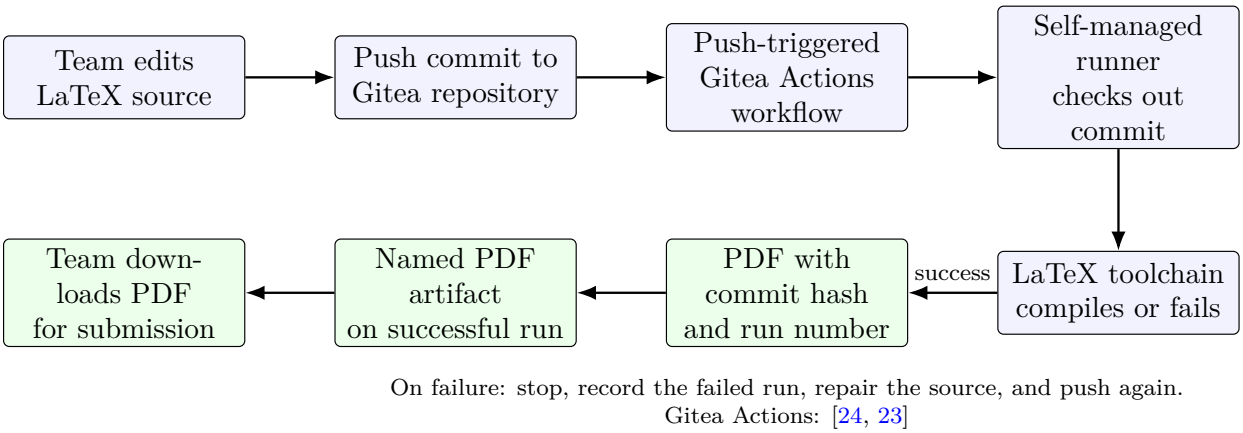


Figure 3.1: From a Gitea source push to a versioned, downloadable manual PDF.

3.2 How the Team Submits Source to Gitea

Team members commit the LaTeX project locally and use a normal `git push` to the repository's HTTPS remote, for example `https://git.example.com/team/manual.git`. The client connects to the public TLS proxy on **TCP port 443**, just as a browser does. The proxy forwards that request on the private container network to Gitea's web service, commonly on **port 3000**. Port 3000 should not be published to the internet merely to allow Git pushes. Set Gitea's public root URL to the HTTPS hostname so its repository links and clone URLs are correct [31, 28].

SSH-based Git pushes are optional and use a *separate* SSH listener: the container's SSH port is commonly 22, while the Droplet may publish it on another port if its own SSH service already uses 22. SSH does not travel through the HTTPS proxy. For this assignment, HTTPS Git pushes are sufficient; teams need not expose a Gitea SSH port. The runner has no public submission port. After a push, Gitea starts the workflow; the runner checks out that revision and uploads the finished PDF to the matching Actions run. The team downloads that PDF artifact and submits *only the PDF* for course grading, as required below [24, 23].

3.3 Where the LaTeX Compiler Runs

Recommended student architecture: keep Gitea as the Git and Actions service, run a separately registered Gitea runner, and let that runner start a temporary *job container* with a pinned TeX Live toolchain. The compiler belongs in the job environment, *not* in the Gitea application container. Gitea does not directly execute LaTeX. A runner label using Gitea's `docker://` execution mode selects the job image; the workflow's `runs-on` value selects that label. A purpose-built image avoids reinstalling TeX on every push [26, 27].

For this course manual, use **TeX Live with `latexmk -pdf`** as the starting point: `latexmk` coordinates repeat passes and `pdflatex` produces the PDF. Include the packages,

fonts, bibliography, glossary, and index tools actually used by the team's project. Record the exact image version or digest and the build command; test it against the manual before relying on Actions. A team needing different font or Unicode support may justify LuaLaTeX instead [62, 11].

The runner registers with and requests work from the Gitea HTTPS URL. For a containerized job, its Docker daemon creates the short-lived TeX job container. The job checks out the triggering Git revision from Gitea over the network, compiles the PDF inside that container, and uploads the PDF back to the workflow run as an artifact. Thus Gitea and TeX exchange source and artifacts through Actions and HTTPS; they need no shared filesystem and no additional public compiler port. Both the runner and the job container must be able to resolve and reach the Gitea URL used for the workflow [27, 23].

A runner and job containers may share the Gitea Droplet if it has enough memory and disk, or the runner may live on a separate machine. Mounting the Droplet's Docker socket into a runner is convenient but gives jobs powerful access to that host and its other containers; restrict who can edit workflows and prefer a separate runner host for stronger isolation. Installing TeX Live directly on a host-mode runner is a workable small project alternative, but jobs then use that runner environment directly and builds are less isolated. If a containerized runner has a `host` label, "host" means *inside the runner container*, not automatically on the Droplet. Installing TeX inside the Gitea container does not configure a runner to use it [26, 27].

Figure 3.2 depicts the recommended single-Droplet student option. Treat the TeX job container as temporary; the Docker daemon and runner socket are internal infrastructure, not internet-facing services.

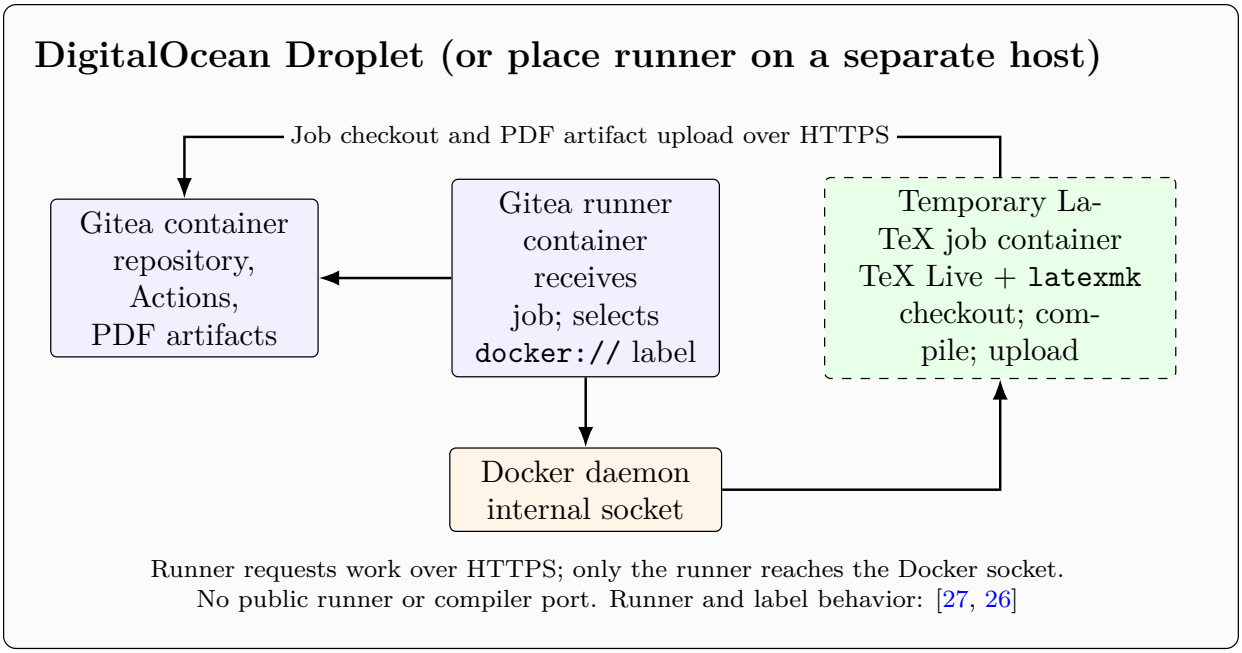


Figure 3.2: Gitea Actions runner launching an isolated LaTeX build job on the same Droplet.

3.4 Required Work

1. Put the complete, buildable LaTeX project, its bibliography, figures, and non-secret build configuration in the team's Gitea repository. Identify the main `.tex` file and a reproducible compile command. The repository is the source of truth for this assignment; editing a PDF does not satisfy the requirement. Push the source over the Gitea HTTPS remote and identify the public and internal ports in the resulting network path.

2. Enable Actions for the repository and register a Gitea runner with a label that the workflow can target. Record its location, version, execution mode, capacity, and registration scope. Ensure the runner and its job environment can reach the Gitea instance. Prefer isolated job execution; do not expose a runner management or Docker daemon port to the internet. Consider available Droplet memory and disk before placing the runner beside Gitea [24, 27].

3. Commit a workflow under `.gitea/workflows/` that runs on a push to the chosen branch. It must check out the triggering revision, prepare a recorded LaTeX toolchain and dependencies, compile the main document, and fail clearly if compilation fails or the expected PDF is absent. Do not download an unreviewed script and execute it with broad privileges. State where the compiler runs and how the job obtains its image or installed tools; do not assume the Gitea container supplies them.

4. Inject the triggering commit hash and workflow run number into the *generated PDF's* visible version information. A small generated LaTeX metadata file or command-line definitions are acceptable. Do not have the workflow commit its generated version back to the same branch: that can create a push/build loop. Keep the manual's normal version and update-history entry under the team's control [25].

5. Upload the successfully compiled PDF as a named Gitea Actions artifact. The build should not silently succeed when the PDF is missing. State how a team member finds and downloads the artifact from the matching run, and record the artifact retention setting if one is configured [23].

6. Demonstrate at least two successive source changes and pushes. For each, correlate the commit, run, visible PDF version, and artifact. Make one controlled LaTeX error on a branch covered by the workflow, then repair it and show a passing run. Do not disrupt the deployed Gitea service.

7. Protect the runner registration token and any repository secrets; do not put them in source, logs, or the PDF. Explain who may edit the workflow and why untrusted workflow changes must not receive privileged Docker access or secrets. Record the runner's access path and any Hosts/Passwords table changes using hints only.

3.5 Exactly What to Update in the IT Manual PDF

Add an *Automated Manual Build* chapter to the team's existing DevOps IT manual. The chapter must have the following eight numbered subsections, in the rubric order. Put dated, labeled, sanitized evidence and clickable Gitea links *inside the PDF*. The workflow and LaTeX source stay in Gitea; submit only the compiled PDF, not those files separately. The final submitted PDF should be downloaded from a successful workflow run.

1. **Source and trigger (15 points).** Link the repository, main `.tex` file, and workflow. Identify the selected branch and push event; show that the workflow runs after a relevant push. State the Git remote protocol, public port, TLS proxy, and private Gitea port; explain why the runner needs no public submission port.

2. **Runner setup (15 points).** Give the runner version, location, scope, label, execution mode, required network path, and capacity. Show a registered/online runner and a job assigned to it. Describe its isolation and absence of unnecessary public ports. Identify whether jobs run in temporary containers or directly in the runner environment.

3. **Reproducible compilation (20 points).** Record the toolchain or build image version, dependencies, build command, main file, and expected PDF path. Link a successful run log and explain how the build detects a compiler error or missing PDF. State where TeX is installed and how the job accesses it; explain checkout and artifact upload without assuming a shared filesystem with Gitea.

4. **PDF version traceability (15 points).** Show the manual's human-maintained version, the generated build identifier, the visible commit hash and run number in the PDF, and the matching Gitea run. Explain how metadata is injected without making an automatic commit or push loop.

5. **Downloadable artifact (10 points).** Link the named PDF artifact on the successful run and show that its downloaded file opens. State the retention setting or observed default; do not treat an artifact as a permanent backup.

6. **Change, failure, and repair evidence (10 points).** Give two successive source commits and matching runs/PDF identifiers, plus one safe failed compile and its repaired passing run. State expected and actual results; match commit IDs across evidence.

7. **Runner and secret boundary (5 points).** Describe workflow edit rights, token/secret storage without values, runner isolation, and any Docker privilege risk. Link the relevant non-secret configuration.

8. **Manual quality and controls (10 points).** Update the manual's version and dated history table; link this chapter and explicitly state whether the Hosts and Passwords tables changed. Keep those tables current, using password hints only. Include relevant citations, glossary/index terms, sanitized evidence, and a short failed-build troubleshooting procedure.

3.6 Rubric (100 Points)

Criterion	Points
Versioned source and push-triggered workflow	15
Registered runner and documented execution environment	15
Reproducible LaTeX compilation and failure detection	20
Visible commit/run version traceability in the PDF	15
Downloadable, verified PDF artifact	10
Two builds plus controlled failure and repair	10
Runner permissions and secret boundary	5
Versioned IT manual, history, tables, and evidence	10
Total	**100**

Chapter 4

Continuous Integration with Jenkins

4.1 Description and Goal

Add automated verification to the same startup product repository established in Chapter 2. A Gitea pull request should trigger a Jenkins pipeline, display a status on the proposed commit, and be unmergeable while the required check fails. After a repair, the team should be able to review and merge it. Use an existing Jenkins container image; building one from source is not required. A small, real test suite is preferable to a long pipeline that proves little. Jenkins and its plugins must be configured as a controlled service, not an unauthenticated public endpoint [43, 42, 48].

4.2 Required Work

1. Deploy an official, recorded Jenkins LTS image using a versioned, non-secret Compose configuration. Persist `/var/jenkins_home`; record the image and installed plugin versions, restart policy, and available Droplet memory/storage. Create named administrator accounts and verify that Jenkins settings and jobs survive a container restart. A backup/restore test is *not* required for this assignment [43].

2. Give Jenkins its own hostname, such as `ci.example.com`, with a valid HTTPS certificate. The existing reverse proxy and a service such as Let's Encrypt may be reused. Set the Jenkins URL and reverse-proxy headers correctly; show certificate validity, redirect behavior, and renewal configuration. Require login and appropriate permissions. Document the DigitalOcean Cloud Firewall, host firewall, and Compose port mappings. Expose only the needed HTTPS entry point; do not publish raw Jenkins port 8080 or an agent port to the internet. Restrict browser access to approved team source ranges or a private access path, in addition to Jenkins login [47, 48]. Provide the complete, working HTTPS URL to the Jenkins login page (for example, `https://ci.example.com/login`), not just a hostname or screenshot. Explain how the instructor can use the approved access path to open that URL without signing in; no grader account is required. This check confirms web-service availability, not that a pipeline works.

3. Integrate the startup's Gitea repository with Jenkins. The Jenkins Gitea plugin and Multibranch Pipeline are a supported route, but another documented, working integration is acceptable. Configure a repository-scoped webhook for relevant push and pull-request events, or explain and demonstrate a polling fallback. If both services are on the same Droplet, use a private container network for webhook delivery where practical; do not open the entire Jenkins interface merely to receive a webhook. Record the event, delivery result, triggering commit, and matching Jenkins run. Use a dedicated Gitea integration identity and only the permissions actually needed for repository access and status reporting [32, 42].

4. Commit a `Jenkinsfile`[1] to the product repository. It must check out the proposed commit, run at least one meaningful build or lint step and one automated test, and report an unambiguous pass or failure. Use the product's actual language/framework. Record where dependencies come from, how a clean run is reproduced, and which stage failed. A command that merely prints "success" is not an adequate test.

   Optional minimal Python application. For this assignment, your team may use the following example as its tested application or use its own application. In either case, commit the application, tests, and `Jenkinsfile` to the team's startup Gitea repository. Create `pricing.py`:

```
def total_price(unit_price, quantity):
    if quantity < 1:
        raise ValueError("Quantity must be positive")
    return unit_price * quantity
```

   Create `tests/test_pricing.py`:

```
import unittest
from pricing import total_price

class PricingTests(unittest.TestCase):
    def test_total(self):
        self.assertEqual(total_price(10, 3), 30)

    def test_invalid_quantity(self):
        with self.assertRaises(ValueError):
            total_price(10, 0)
```

   For this Python example, the following complete `Jenkinsfile` uses a Declarative Pipeline. Commit it at the root of the same Gitea repository:

[1] Create this plain-text file at the root of your startup product's Gitea repository and commit it with the code. It is the pipeline recipe Jenkins runs for a pull request: use real build or lint and test commands, then verify the run and its reported status. Link to the file and matching run in your IT manual PDF; do not submit the file separately.

```
pipeline {
    agent any
    options { skipDefaultCheckout() }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Syntax check') {
            steps { sh 'python3 -m py_compile pricing.py' }
        }
        stage('Test') {
            steps { sh 'python3 -m unittest discover -s tests -v' }
        }
    }
}
```

The selected Jenkins executor must have `python3` available; it need not be installed in the Gitea container. `checkout scm` fetches the source revision selected by the Jenkins job from Gitea into the executor's workspace, so verify that the run and reported Gitea check correspond to the pull request's latest head commit. With a separate trusted build-agent container, the agent needs Git, Python, and private network access to Gitea; Jenkins supplies any required repository credentials. No Docker daemon or Docker socket is needed merely to run these example checks. Replace `agent any` with the agent's configured label, such as `agent { label 'python' }`. This file runs the checks; it does *not* by itself configure a Gitea webhook, publish a status, or protect `main` [45, 49]. To demonstrate Step 6, replace the multiplication with addition on the feature branch and push it: the test must fail and the required check must block merging. Restore the multiplication, push a corrective commit, and show the passing check before review and merge. If you choose this example, it counts as your application for this assignment, but you must still demonstrate the issue-linked pull request, Jenkins status, protected-branch merge gate, and PDF evidence required below.

5. Protect the default branch in Gitea and require the Jenkins status check on the pull request head commit before merge. Identify the exact check context; do not confuse a successful webhook delivery or a green Jenkins UI page with a passing merge gate. Show that a failed check blocks merging and that a passing check permits the team's normal peer-review process. The Gitea Checks plugin is one way to publish status, but a proven equivalent is acceptable [29, 41].

6. Demonstrate a controlled failure and repair on one issue-linked pull request: show the failed commit, failed test/run, blocked merge, corrective commit, passing run/status, peer review, and eventual merge. Use real links and matching commit identifiers so the sequence can be audited. Do not intentionally break the deployed Gitea or Jenkins services.

7. Keep tokens and passwords in Jenkins credentials or an approved secret store, never in the repository, Compose file, console log, or report. Keep credential scope narrow and explain who can change the `Jenkinsfile`. Pipeline code executes commands. For this assignment, teams may run builds on a Jenkins controller executor or in a separate build-agent container; the latter is recommended but not required. State the choice and show the actual build location in the adapted deployment diagram. If using an agent, set controller executors to zero and show its private connection to Jenkins and its checkout path to Gitea. If building on the controller, explicitly document the risk, restrict pipeline authors to trusted team members, and do not grant untrusted pull requests access to secrets or privileged Docker access. Record a plugin-update check and the operator response to a failed webhook or build [40, 39].

Example for Step 6: Open an Issue-Linked Pull Request

First, create an issue in the startup repository using Gitea's *Issues* → *New Issue* page and note its number. This example uses issue #42 and the optional Python application above; substitute your actual issue number and changed filenames. In a local clone[2], run:

```
git switch main
git pull --ff-only origin main
git switch -c feature/issue-42
git add pricing.py tests/test_pricing.py Jenkinsfile
git commit -m "Test pricing change for issue #42"
git push -u origin feature/issue-42
```

In Gitea, open *Pull Requests* → *New Pull Request* (or follow the compare link shown after the push). Select `main` as the *target/base* branch and `feature/issue-42` as the *source/head* branch. Give the pull request a descriptive title and put `Closes #42` in its description. The branch name alone does not link the issue; the description's closing reference does, and Gitea closes the issue when the pull request is merged. Request a teammate's review. Make the controlled repair on the same feature branch, commit it, and run `git push` again; Gitea updates the existing pull request. Merge only after the latest head commit passes the required Jenkins check and receives the team's peer review [30, 19, 29].

4.3 Activity Diagram: Pull Request to Merge

Figure 4.1 shows the workflow students must implement. The webhook starts a Jenkins run, but the merge gate depends on Jenkins reporting the required check context to Gitea for the pull request's latest head commit. A failed or missing check blocks the merge. A successful check permits peer review; it does not replace peer approval. Each corrective push creates a new head commit that must be tested and reported again.

[2] A local clone is a copy of the Gitea repository on your computer. On the repository page in Gitea, copy its HTTPS clone URL, then run `git clone <that-URL>` in a terminal and change into the new repository directory with `cd <repository-name>`. Run the commands below from that directory.

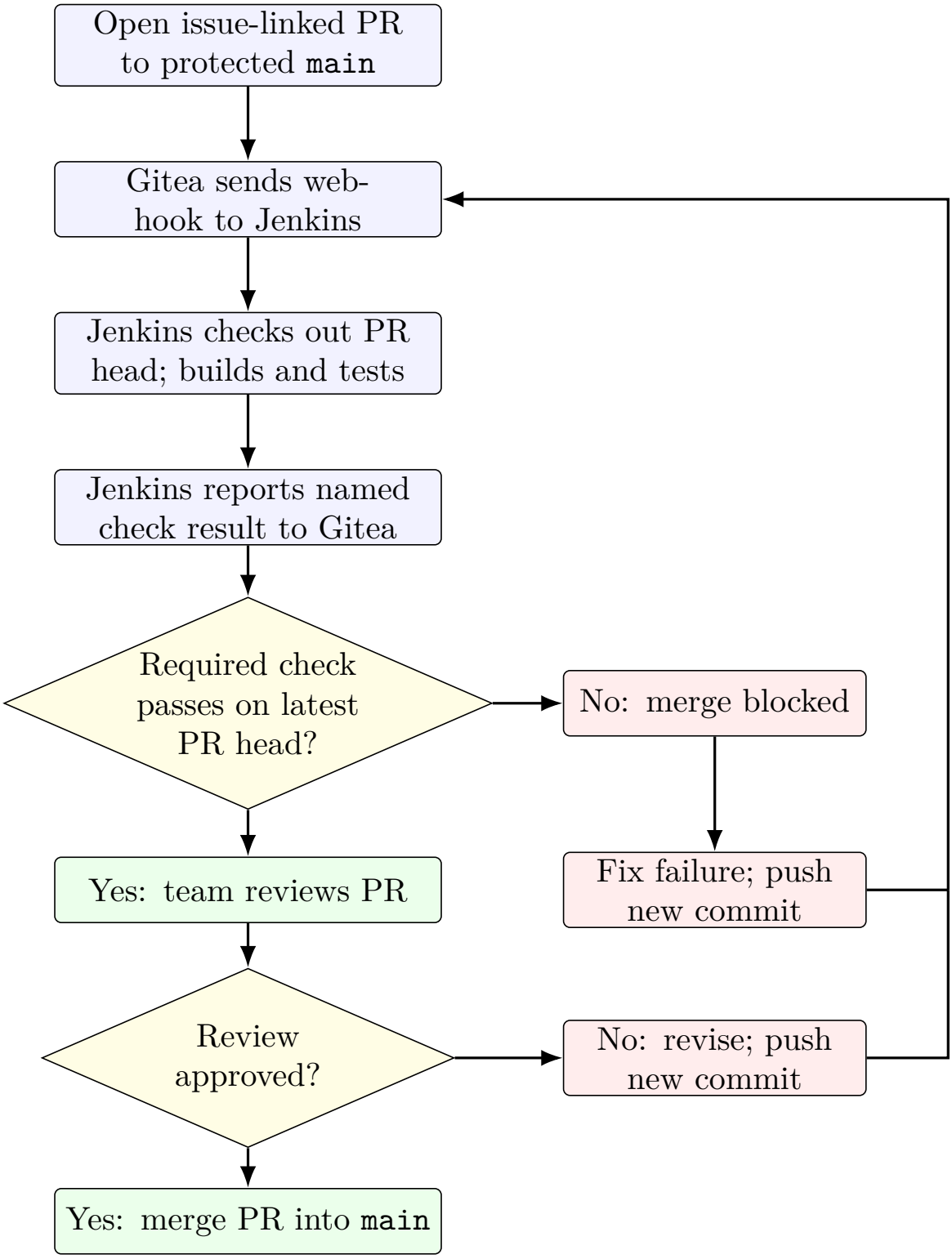


Figure 4.1: Jenkins–Gitea pull-request activity. A new commit repeats the build; the reported check on the latest head commit, not webhook delivery, gates merging. Sources: [29, 41].

4.4 Network and Trust Boundary

Show browser-to-proxy-to-Jenkins and Gitea-to-webhook paths separately. A private webhook path does not make the browser interface private, and HTTPS does not replace firewall rules or authentication. If Gitea and Jenkins are on different Droplets, use a documented private network or a narrowly reachable HTTPS webhook endpoint rather than exposing port 8080. Explain which source addresses can reach each path. Check public exposure from outside the Droplet over IPv4. IPv6 is optional; if enabled on the Droplet or published in DNS with an AAAA record, check its firewall exposure too. Before narrowing SSH or browser access, verify an approved administrator can still connect. Do not publish an inbound agent port solely for this assignment [32, 47].

Figure 4.2 is a reference deployment, not a prescribed implementation. It shows the same-Droplet option with builds on the Jenkins controller executor. Teams using separate Droplets must draw the actual network boundary and webhook route. Replace the Gitea data node with the SQLite volume or PostgreSQL service and volume chosen in Chapter 2. If deploying a separate build-agent container, add it to the adapted diagram and show both its private controller connection and its direct Gitea checkout path.

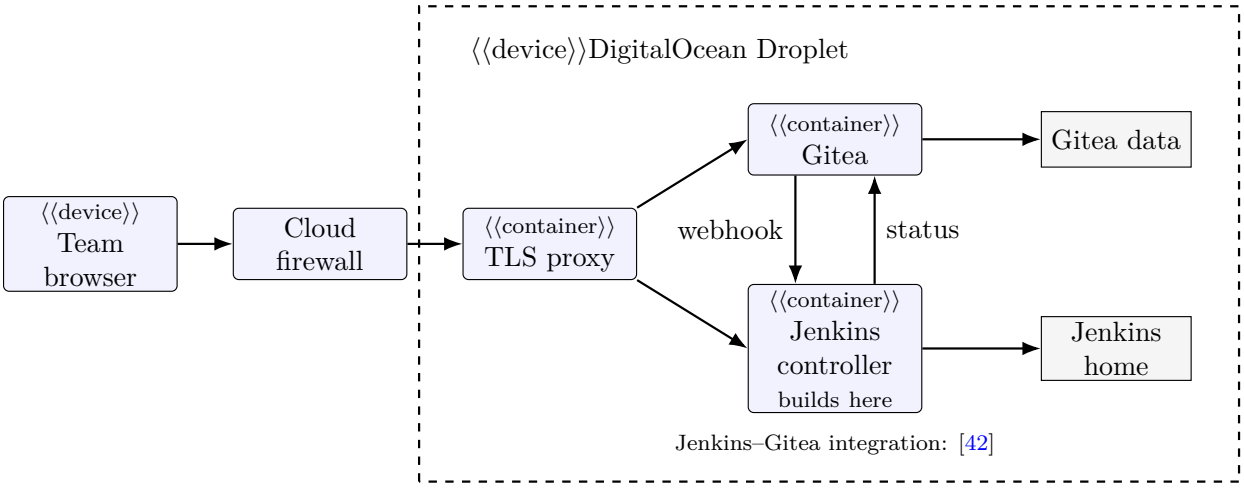


Figure 4.2: UML deployment reference for one-Droplet Gitea and Jenkins integration. The browser reaches the TLS proxy through firewall-approved HTTPS (443). Webhook, repository checkout, status, and data paths stay private. Builds in this reference run on a Jenkins controller executor. The Gitea data node represents the team's chosen database and persistent repository storage.

4.5 Exactly What to Update in the IT Manual PDF

Submit *only* the updated, compiled PDF of the team's existing LaTeX DevOps IT manual. Add a *Jenkins/CI* chapter with the following eight numbered subsections, in the same order as the rubric. Include labeled, dated, sanitized evidence and clickable links *inside the PDF*; keep the versioned `Jenkinsfile` and non-secret deployment configuration in Gitea and link

to them. Do not submit those files separately. The written chapter is the primary deliverable: explain what the team configured, why, how it was verified, each test's expected and actual result, and any failure and correction. Use a small number of labeled screenshots, redacted excerpts, and links to support those explanations; a screenshot without an explanation is not sufficient. No video is required.

1. **Jenkins deployment and persistence (10 points).** Record the official image source and tag, plugin versions, Compose link and redacted excerpt, Droplet capacity, persistent volume, restart policy, and container restart test. Show the Jenkins configuration and job still present afterward. State explicitly that restart persistence is not a backup test.

2. **HTTPS, access, and firewall (15 points).** Provide a clickable, complete HTTPS URL to the live Jenkins login page, not only a hostname or screenshot. Explain the approved access path the instructor must use to open it; no sign-in is required. Include a dated, labeled screenshot as supporting evidence. The login page demonstrates web availability only. Show DNS, certificate issuer/expiry, HTTP-to-HTTPS behavior, renewal configuration, Jenkins URL/proxy settings, named accounts and roles, Cloud Firewall rules, and host/Compose port mappings. Give dated external test results for HTTPS, raw port 8080, and any agent port over IPv4, and identify the approved browser-access path. IPv6 is optional: if enabled on the Droplet or published in DNS, include the equivalent IPv6 checks; otherwise state that the deployment uses IPv4 only.

3. **Gitea trigger and traceability (15 points).** Adapt Figure 4.2 as a labeled UML deployment diagram of your actual browser and webhook paths. Identify the Gitea repository, integration identity, event selection, webhook/polling configuration, delivery result, commit ID, and linked Jenkins run. Show where checkout and tests run; if using a separate build-agent container, include its controller and Gitea paths. Explain one failed delivery diagnosis or the steps the operator would take to diagnose one.

4. **Versioned pipeline and tests (15 points).** Link the `Jenkinsfile`; describe checkout, build/lint, and automated-test stages, dependencies, expected outputs, and clean-run instructions. Include a sanitized run view with distinct stage results and the tested commit ID.

5. **Required merge gate (15 points).** Show the default-branch protection setting, exact required status-check context, failed check blocking a pull-request merge, and passing check allowing the usual review and merge. Include two labeled screenshots in the PDF: one showing the failed check blocking the merge and one showing the passing check after repair. In their captions, identify the date, commit ID, and check context; link to the pull request and matching Jenkins runs. Explain how the status reaches Gitea from Jenkins.

6. **Failure, repair, and review (10 points).** Give an issue and pull-request link and a dated sequence of failed commit/test, blocked merge, repair commit, passing run/status, peer review, and merge. Match commit IDs across Gitea and Jenkins.

7. **Credentials and build boundary (10 points).** Identify secret storage and scope without revealing values; state who can edit the `Jenkinsfile`, where builds execute, how

untrusted contributions are kept from secrets, and the plugin-update check. Include a short webhook/build troubleshooting procedure. A controller executor is permitted if its specific risk and compensating restrictions are documented. If using a separate agent, show that controller executors are zero and explain how the agent retrieves code from Gitea.

8. **Manual quality and controls (10 points).** Write a coherent implementation narrative with procedures, decisions, expected versus actual results, and corrective actions (4 points). Provide readable, dated, labeled, sanitized screenshots and traceable links that support the narrative, including the two merge-gate states above (3 points). Increase the PDF's version; add a dated history entry linking this chapter and naming updates to the Hosts and Passwords tables. Update both tables (password *hints only*), the UML deployment diagram, relevant citations, glossary/index entries, and an operator restart/troubleshooting note (3 points). Redact all secrets.

4.6 Rubric (100 Points)

Criterion	**Points**
Jenkins deployment and persistence	10
Working HTTPS login-page URL, access control, and IPv4 firewall	15
Gitea trigger and traceable Jenkins run	15
Versioned pipeline with meaningful build and tests	15
Required Gitea status check and merge gate	15
Failed-then-repaired, reviewed pull request	10
Credentials, pipeline trust boundary, and plugin operations	10
IT manual narrative (4), supporting evidence (3), and document controls (3)	10
Total	**100**

Chapter 5

Introduction to SRE: Monitoring and Reliability

5.1 Description and Goal

Your startup can now host its work in Gitea, build its IT manual automatically, and use Jenkins to test changes before merging them. The next operational question is: *How will the team know that this delivery platform is working, notice degradation, and restore service when something fails?* This assignment introduces site reliability engineering (SRE) through the infrastructure you already operate. Add Prometheus to collect time-series metrics and Grafana to query, visualize, and alert on those metrics. Use measured evidence to explain the reliability of Gitea, Jenkins, and their shared DigitalOcean Droplet.

SRE applies engineering to operating reliable services. This chapter practices monitoring, a service-level indicator (SLI), a service-level objective (SLO), actionable alerts, a runbook, and a blameless incident review. It is an introductory exercise rather than a complete SRE program. A dashboard alone is insufficient: the team must show that its measurements describe a real condition and lead to a useful response. The SLO approach follows the Google SRE Workbook [35].

5.1.1 How This Assignment Extends the Earlier Work

Existing capability	What you add now
Gitea foundation (Chapter 2)	Observe the existing HTTPS service and track host capacity. Retain its repositories, identities, TLS, and data.
Manual build (Chapter 3)	Expand the same team IT manual and publish its updated PDF through the established Gitea Actions workflow.
Jenkins CI (Chapter 4)	Observe the existing controller and pipeline; correlate build activity with metrics, and distinguish a failed build from an unavailable Jenkins service.
This assignment	Add persistent monitoring, an availability objective, tested notifications, and a practiced response to a controlled outage.

Starting point. Begin with working Gitea and Jenkins HTTPS hostnames, the existing issue/branch/pull-request workflow, a passing Jenkins pipeline, and a successful manual-PDF build. Repair unresolved prerequisites before collecting the baseline. Use the same team repository and droplet; no Kubernetes cluster, new product application, or paid monitoring service is required. If capacity requires a different deployment, document and justify it, including any additional cost.

Completion criteria. Another team member should be able to follow your manual to deploy the monitoring configuration, interpret the dashboard, receive an alert, restore Jenkins, and explain what the measurements do and do not prove. Keep all earlier capabilities working at the end.

5.2 Deployment and Tool Responsibilities

Component	Required responsibility
Prometheus	Scrape and retain timestamped metrics; provide the query API used by Grafana.
Grafana OSS	Present the dashboard and evaluate Grafana-managed alert rules against Prometheus; send notifications to an approved test destination.
Node Exporter	Expose whole-droplet CPU, available memory, and root-filesystem space. These measurements combine all host workloads; they do not identify individual containers or processes.
Blackbox Exporter	Probe the existing Gitea and Jenkins HTTPS URLs and expose probe success and duration.
Gitea and Jenkins	Remain the existing work and CI services; Blackbox Exporter checks their HTTPS endpoints without installing application plugins.

Container deployment. Run four separate containers on the existing Linux droplet: Prometheus, Grafana OSS, Node Exporter, and Blackbox Exporter. Use Docker Compose and images published by the projects; students do not build these images or write Dockerfiles. Keep the existing Gitea and Jenkins installations and integrate them with this monitoring stack. Each blue box in Figure 5.1 represents one new container.

Figure 5.1 is a reference deployment, not a finished configuration. Adapt it to your actual hostnames, networks, processes, and volumes. Arrows show who initiates a request; metric responses return along the same connection. Exporters are metric endpoints, not databases. Grafana queries Prometheus rather than scraping Gitea or Jenkins itself.

Scope and limitations. Use Grafana-managed alerts; a separate Alertmanager is not required. Centralized logs, traces, Kubernetes, high availability, and automated remediation are optional future extensions. cAdvisor and per-container/process resource monitoring are outside the required scope. Monitoring on the same droplet shares its failure domain: a total droplet outage can also stop collection and notifications. Identify this limitation and propose an independent external check in your review. A local HTTPS probe does not prove reachability from every user network.

5.3 Required Implementation Steps

Complete the steps in order. Record actual commands, selected versions, configuration locations, test times, and results in your team manual. Replace example hostnames, labels, and paths with values verified in your deployment.

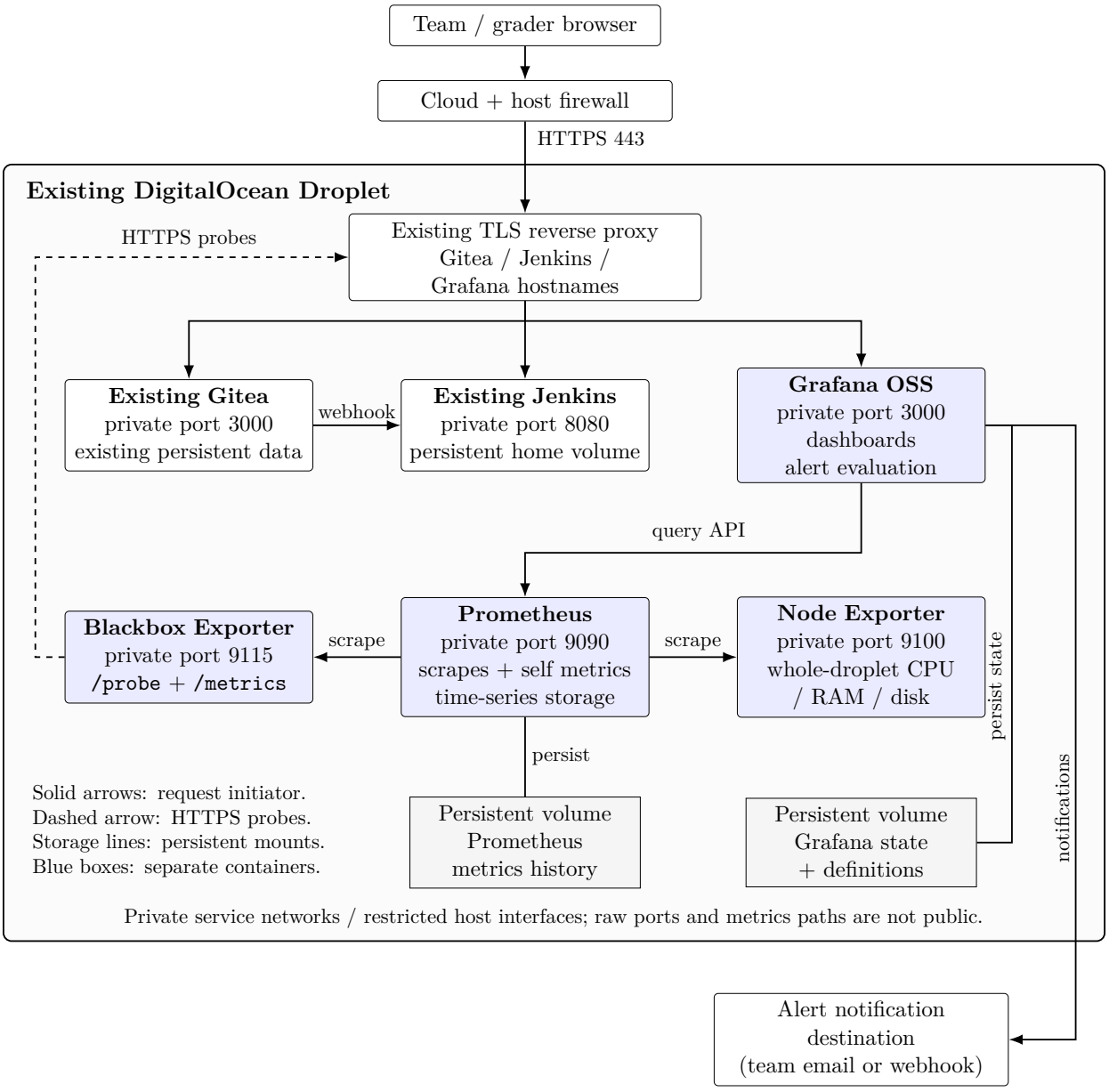


Figure 5.1: Existing Gitea/Jenkins deployment extended with monitoring. Each blue component is a separate container. All connectors use horizontal and vertical segments. Internal ports are examples, not public firewall allowances. Verify the probe's actual hostname/DNS route. Based on the official component documentation [57, 36, 56, 54].

5.3.1 Step 1: Establish a Baseline and Plan the Change

1. Open a Gitea issue, create a feature branch, and assign an implementation owner and a second team member to verify it. Record the starting commit and a passing Jenkins build/manual build.

2. Record the droplet's CPU, RAM, free disk, OS, services, and software versions. Observe resource use during one normal Jenkins build. Explain whether the host has headroom for monitoring, builds, and service data.

3. Choose compatible, fixed releases or image digests for the new four monitoring components. Consult documentation for those releases; do not silently upgrade existing services to make an example work. State retention and storage limits. Seven days is a reasonable lab starting point if capacity permits; preserve room for the operating system and builds.

4. Draw the intended deployment and write a recovery plan before changing services. Identify how to restore the previous configuration and who will restart Jenkins during the outage exercise.

5.3.2 Step 2: Deploy Persistent, Private Monitoring

1. Add a Docker Compose file, configuration files, and provisioning files to Gitea, using the installation guide below. Record startup order, identities, mounts, volume paths, and restart behavior. Validate configuration before starting or reloading it.

2. Give Prometheus a persistent time-series data volume and Grafana persistent application storage. Retain dashboards, data-source definitions, and alert rules through provisioned files or versioned sanitized exports. Document permissions and a backup procedure for state that cannot be reconstructed from Git [55, 38].

3. Put raw interfaces on private service networks or restricted host interfaces. Do not publish Prometheus 9090, Node Exporter 9100, Blackbox Exporter 9115, or application backend ports to the Internet. Do not give monitoring containers the Docker socket. Identify and justify any read-only host mounts or namespace permissions required by an exporter.

4. Add authenticated Grafana behind your existing TLS proxy, using a dedicated hostname such as `metrics.team.example`. Disable anonymous dashboard access and replace initial credentials. Preserve Gitea/Jenkins HTTPS routing. Gitea and Grafana may both listen on port 3000 in *different* containers; use distinct service names and proxy upstreams, not conflicting host mappings.

5. Align cloud firewall, host firewall, Docker networking, and proxy routes. No Gitea or Jenkins metrics endpoint is required. If you enable optional application metrics, block those routes externally, including through the HTTPS hostnames. Retain earlier SSH restrictions and HTTP policy. From an external machine, test intended HTTPS login pages and blocked raw ports/metrics paths over IPv4, and IPv6 if enabled. Record source, commands, results, and date; firewall screenshots alone do not prove exposure.

Installation Guide: Use Published Images

Use the project-published repositories below. A registry hosts images; the publisher and its official documentation establish the source. Do not choose an unrelated community image merely because its name resembles the tool. Replace every `CHOOSE_TAG` with a specific supported release tag verified for the droplet's CPU architecture; never leave `latest` in the submitted deployment. Record the pulled digest as well as the tag.

Container	Project-published image repository
Prometheus	`prom/prometheus` (Docker Hub) [55]
Grafana OSS	`grafana/grafana` (Docker Hub) [38]
Node Exporter	`quay.io/prometheus/node-exporter` [56]
Blackbox Exporter	`quay.io/prometheus/blackbox-exporter` [54]

Use `grafana/grafana` for OSS; the older `grafana/grafana-oss` repository is no longer updated beginning with Grafana 12.4.0. Follow the current official installation documentation [38].

1. Prepare the droplet. First run `docker version`. Then check the Compose plugin with `docker compose version`. If missing, install Docker Engine and the Compose plugin using Docker's instructions for the droplet's actual Linux distribution, linked from https://docs.docker.com/engine/install/. Do not reinstall or reset a working Docker installation. Create a monitoring directory in the team's repository, with `compose.yaml`, `.env`, `prometheus/prometheus.yml`, and `blackbox/blackbox.yml`. The following `.env` contains version choices only, not passwords:

```
PROMETHEUS_TAG=CHOOSE_TAG
GRAFANA_TAG=CHOOSE_TAG
NODE_EXPORTER_TAG=CHOOSE_TAG
BLACKBOX_TAG=CHOOSE_TAG
```

2. Define the four containers. Put this starting configuration in `compose.yaml` on the Linux droplet. Read-only mounts provide configuration; named volumes retain state. The shared bridge provides service-name DNS. Only Grafana is published, on host loopback port 3001, for a reverse proxy running on the host. Its internal port remains 3000.

```yaml
services:
  prometheus:
    image: prom/prometheus:${PROMETHEUS_TAG:?choose a release}
    restart: unless-stopped
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=7d
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
```

```
    networks: [monitoring]

  grafana:
    image: grafana/grafana:${GRAFANA_TAG:?choose a release}
    restart: unless-stopped
    ports: ["127.0.0.1:3001:3000"]
    environment:
      GF_AUTH_ANONYMOUS_ENABLED: "false"
      GF_USERS_ALLOW_SIGN_UP: "false"
    volumes:
      - grafana_data:/var/lib/grafana
    networks: [monitoring]

  node-exporter:
    image: quay.io/prometheus/node-exporter:${NODE_EXPORTER_TAG:?choose a release}
    restart: unless-stopped
    pid: host
    command:
      - --path.rootfs=/host
      - --path.procfs=/host/proc
      - --path.sysfs=/host/sys
    volumes:
      - /:/host:ro,rslave
    networks: [monitoring]

  blackbox-exporter:
    image: quay.io/prometheus/blackbox-exporter:${BLACKBOX_TAG:?choose a release}
    restart: unless-stopped
    command: ["--config.file=/config/blackbox.yml"]
    volumes:
      - ./blackbox/blackbox.yml:/config/blackbox.yml:ro
    networks: [monitoring]

networks:
  monitoring:
volumes:
  prometheus_data:
  grafana_data:
```

The Node Exporter example supplies the host filesystem, procfs, sysfs, and PID namespace for this assignment's CPU, RAM, and disk measurements. The read-only root mount still exposes host files to that container; do not add `privileged: true` or mount the Docker socket. This bridge-network variant does not share the host network namespace. Do not claim that its network-interface metrics represent all host interfaces. The official recipe uses host networking for that broader view; changing to it requires a private listen address, a reachable Prometheus target, and fresh exposure checks [56]. Compare CPU counters, memory totals, and root filesystem size/free space with `/proc/stat`, `free -b`, and `df -B1 /` on the droplet before accepting the deployment.

3. Add minimal configuration. Start `prometheus/prometheus.yml` with the following complete set of required jobs. Replace both example HTTPS URLs with your actual login

URLs before startup:

```
global:
  scrape_interval: 30s
scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]
  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]
  - job_name: blackbox
    static_configs:
      - targets: ["blackbox-exporter:9115"]
  - job_name: https
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - https://git.team.example/user/login
          - https://ci.team.example/login
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115
```

Use this initial `blackbox/blackbox.yml`; an HTTP module alone does not schedule probes; the `https` job above schedules them. Step 4 verifies that the probe results describe your actual services.

```
modules:
  http_2xx:
    prober: http
    timeout: 10s
    http:
      method: GET
      preferred_ip_protocol: ip4
      tls_config:
        insecure_skip_verify: false
```

The IPv4 preference makes the initial lab route explicit; document the actual address family and add IPv6 checks if enabled. Adapt expected HTTP responses and redirects to the team's real HTTPS login URLs.

4. Pull, validate, and start. Run these commands from the monitoring directory after choosing release tags and creating both files:

```
docker compose config --quiet
docker compose pull
docker compose run --rm --no-deps --entrypoint /bin/promtool \
  prometheus check config /etc/prometheus/prometheus.yml
docker compose up -d
docker compose ps
docker compose logs --tail=50 prometheus grafana node-exporter blackbox-exporter
docker compose images
```

The YAML validation checks Compose structure; `promtool` checks Prometheus configuration. Inspect logs and actual target health to establish that all four services work. Record each image's `RepoDigests` using `docker image inspect IMAGE:TAG`; review logs before including them in the PDF. No `docker build` is required.

5. Integrate the existing services. For a host-based proxy, route Grafana's HTTPS hostname to `127.0.0.1:3001`; complete initial login through HTTPS and immediately replace the initial administrator password. Set Grafana's external root URL to the real HTTPS hostname. If the proxy is containerized, remove the loopback port mapping and attach Grafana to the proxy's existing Docker network; use `grafana:3000` as its upstream. Declare that network as external in Compose so it is reused, not recreated. Keep Prometheus and exporters on the monitoring network only.

Configure Grafana's Prometheus data source as `http://prometheus:9090`. A localhost address would refer to the Grafana container. The required setup needs no direct metrics connection to Gitea or Jenkins. Blackbox Exporter reaches their existing HTTPS hostnames through the reverse proxy. Verify DNS resolution and routing from the monitoring network rather than assuming a browser test proves this path. Keep normal bridge egress for HTTPS probes and approved notifications.

6. Preserve and maintain the installation. Commit the Compose and non-secret configuration files, selected versions, and sanitized exports. Keep credentials outside Git. Use `docker compose stop` and `docker compose start` for routine pauses; do not use `docker compose down -v`, which deletes the named data volumes. For an upgrade, back up state, review release notes, change one pinned image, pull and recreate that service, then verify health and persistence. Document rollback compatibility rather than assuming older software can read upgraded data. The later steps still require host metrics, probes, dashboards, alerts, and incident evidence; starting this scaffold alone is incomplete.

5.3.3 Step 3: Verify Host Metrics and Collection Health

1. Verify the required targets: Prometheus itself, Node Exporter, and Blackbox Exporter's own metrics endpoint. Retain the 30-second scrape interval unless you document a reason to change it. Show successful scrapes using the private target view or an `up` query in

Grafana Explore. An exporter being reachable does not establish that its measurements are correct.

2. Compare Node Exporter's memory totals and available memory with `free -b`; compare root filesystem size and available space with `df -B1 /`. Identify the actual host root filesystem's `mountpoint`, `device`, and `fstype` labels. With the host root mounted at `/host`, inspect the exported labels rather than assuming the root label is `/`. Exclude overlay and pseudo filesystems.

3. Build a small inventory covering CPU, available memory, root disk, `up`, `probe_success`, and `probe_duration_seconds`: record exported names, type, unit, labels, meaning, and limitation. CPU time is a counter; derive utilization from its rate rather than graphing its raw lifetime total. Memory available is an estimate of memory available to applications, not simply unused RAM.

Optional extension, not required for credit. Teams may add private Gitea metrics or the Jenkins Prometheus plugin after completing the core exercise [21, 46]. This requires additional configuration, credentials, networking, and exposure checks. Neither plugin installation nor an application-metrics dashboard panel is required.

5.3.4 Step 4: Measure HTTPS Availability with Probes

Verify the supplied Blackbox Exporter module and Prometheus job named `https` for the actual Gitea and Jenkins HTTPS login URLs. Use the multi-target pattern: pass each URL as the probe target, preserve it as the `instance` label, and direct the scrape to Blackbox Exporter's `/probe` endpoint. Also scrape its own `/metrics` endpoint as a separate job. Keep the probe interface private.

Define success explicitly: DNS resolution, valid TLS certificate/hostname, allowed redirects, and the expected final HTTP response. Do not disable TLS verification. Document whether a login-page response or a content check is used. A successful login-page probe does not demonstrate authenticated Git operations or a working build. Record `probe_success` and `probe_duration_seconds` for both URLs. The exporter's `up` can be 1 while a target's `probe_success` is 0: collection worked, but the probed request failed [54].

5.3.5 Step 5: Build a Dashboard That Answers Operational Questions

Connect Grafana to Prometheus through its private service address. Create a named team dashboard with an explicit time zone, readable units, legends, thresholds, and a useful default time range. Explain each query; imported dashboards must be adapted and attributed [36].

Include these six required panels:

1. Whole-droplet CPU utilization (percent, averaged across CPUs).

2. Available host memory (bytes or percent; identify the chosen unit).

3. Available space on the actual host root filesystem (percent).

4. Gitea HTTPS availability (`probe_success`, 0 or 1).

5. Jenkins HTTPS availability (`probe_success`, 0 or 1).

6. HTTPS probe duration for both services (seconds, separate labeled lines).

Show scrape health in a separate target view or an optional additional panel. For the root disk panel and alert, use available bytes divided by size bytes for the same verified filesystem. Keep selectors specific to this droplet and filesystem. Do not combine unrelated mounts into one percentage.

Capture host metrics before, during, and after one normal run of the existing Jenkins pipeline. Annotate build ID, commit, start/end timestamps, and where the build actually executed (controller, local agent container, or remote agent). Explain an observed change or why none was visible: short builds may fall between scrapes, and a remote agent's resource use is outside this host's measurements. Do not add a new application or synthetic load solely to force a visible spike. Whole-host correlation does not attribute CPU or RAM to Jenkins or another individual container. Preserve the dashboard definition in Gitea; screenshots alone are not recoverable configuration.

5.3.6 Step 6: Define and Evaluate One Availability Objective

Term	Meaning in this assignment
SLI	A measured indicator: the fraction of scheduled HTTPS probes that succeed for a specified service and window.
SLO	An internal target: at least 99% success over a 24-hour lab observation window, evaluated separately for Gitea and Jenkins.
SLA	A service-level agreement with a customer, potentially including consequences for a missed commitment. This exercise establishes no customer SLA.

Required: explain a worked example. Use this hypothetical, complete dataset: each service has 2,880 scheduled probes over 24 hours at a 30-second interval. Gitea has 2,880 successes and no failures; Jenkins has 2,850 successes and 30 failures. There are no missing observations. Gitea's sample success percentage is 100%; Jenkins's is $100 \times 2850/2880 \approx 98.96\%$, which misses the 99% target. The theoretical budget is $0.01 \times 2880 = 28.8$ failed samples, or 14.4 minutes under uniform sampling. Thirty failures exceed that budget by 1.2 samples (about 0.6 minutes); at most 28 whole failed samples would meet the target. Explain these calculations in your own words and distinguish an SLI, an SLO, and an SLA. These are illustrative numbers, not your lab results.

Required: interpret your actual exercise. Select a short recorded window spanning the controlled Jenkins outage and recovery. State its exact start/end, time zone, scrape

interval, and query evaluation time. Show `probe_success` for both services and explain what it demonstrates. For example, a 10-minute query is:

```
100 * avg_over_time(
  probe_success{
    job="https",
    instance="https://ci.team.example/login"
  }[10m]
)
```

Use `count_over_time` on the same series and window to report received samples, and compare with approximately 20 expected at 30 seconds. Explain any missing samples or boundary differences. This percentage uses only received observations: missing data is unknown and must not be counted as success. A short exercise cannot establish the 24-hour SLO result [58].

Optional: evaluate the full-day SLO. Collect a complete 24-hour window and calculate per-service success percentages, expected/received sample counts, coverage, and budget consumed/remaining. Explain how missing observations affect the conclusion; do not infer downtime from missing collection alone. Full-day collection is not required for the 100-point rubric. The 99% lab target is an instructional choice, not a production reliability promise.

5.3.7 Step 7: Configure and Test Two Actionable Alerts

Create two Grafana-managed rules. Give each an owner, severity, query, threshold, evaluation interval, pending period, summary, and runbook link. Choose explicit behavior for missing data and evaluation errors that brings monitoring failure to the team's attention rather than silently declaring health. Document notification grouping/delay settings [37].

1. **Service unavailable:** alert per service when its HTTPS probe continues to fail for at least one minute. Account for scrape, evaluation, pending, and notification delays. Ensure an absent series or failed collection is also surfaced by no-data/error handling or an additional collection-health rule; do not treat it as success.

2. **Host capacity:** alert when available space on the host root filesystem stays below 15% for five minutes. Select the actual host filesystem and exclude container/pseudo filesystems. Explain the threshold and the operator's response.

Route both to an approved team-controlled destination such as a test email address or webhook. Test the contact point, then demonstrate real rule evaluation, firing, notification delivery, and resolution for each rule. A contact-point test alone is insufficient. Test capacity by *temporarily raising the free-space threshold above the current value*, waiting through the pending period, then restoring 15% and verifying resolution. Do not fill the disk. Record the temporary change and restore all final settings. Redact destination secrets.

Set up the notification destination. The alert notification destination in the deployment diagram is where Grafana sends alerts to the team; it is not another required monitoring container. One contact point shared by both rules is sufficient. Choose one of the following routes:

Webhook or chat integration.
Use an approved team-controlled HTTPS receiver that accepts Grafana's webhook payload. This route avoids SMTP configuration. For a supported chat service, select its dedicated Grafana integration rather than assuming that a generic webhook's JSON matches the service's expected format. Supply the receiver URL and any required authentication. Grafana needs outbound access to the destination; the receiver does not need access to Grafana's private service ports.

Email.
Grafana OSS needs an SMTP service before an Email contact point can send messages. Obtain the provider's SMTP host/port, username, password or app credential, permitted sender address, and TLS requirements. Configure the `[smtp]` section in Grafana's mounted `grafana.ini`, or use the equivalent container environment settings. Restart Grafana after changing its configuration; recreate the Grafana container if changing its Compose environment. Preserve its named volume. Then enter the team's recipient address in the Email contact point. The recipient address alone does not configure outgoing mail. Follow the official Grafana OSS email setup guide.

Create, connect, and verify the contact point. Menu names vary by Grafana version; use the documentation for the installed version. The current workflow is:

1. Open *Alerting*, then *Notification configuration / Contact points* (some versions show *Alerts & IRM* first). Use the built-in Grafana Alertmanager for these Grafana-managed rules.

2. Create a contact point named `team-alerts`. Select Email, Webhook, or the appropriate dedicated integration, and fill in its required settings. Leave resolved notifications enabled.

3. Use *Test / Send test notification*, confirm receipt at the actual destination, and save the contact point.

4. In each of the two alert rules, select `team-alerts` in its notification settings. If the installed version uses notification policies instead, route each rule's labels to this contact point and check that both rules match. Record the grouping and delay settings actually used.

5. Perform the controlled rule tests described above and the Jenkins incident in Step 8. Capture timestamped firing and resolved messages; a successful contact-point test verifies delivery, not the queries, thresholds, or rule-to-destination routing.

See the official contact-point instructions. Keep SMTP credentials, authentication tokens, and private webhook URLs out of Gitea and submitted screenshots. Document the configuration structure with placeholders and explain how the team supplies secrets locally.

5.3.8 Step 8: Run a Controlled Incident and Verify Recovery

1. Agree on a short team maintenance exercise. Check that no build is running, retain the recovery command, and start a timestamped incident record. Keep Gitea and monitoring available.

2. Stop only Jenkins for approximately three minutes, long enough for your configured outage alert to notify. Do not destroy volumes or stop the droplet. Restore promptly if the exercise exceeds its agreed duration or affects unrelated work.

3. Record injection, first failing probe, alert firing, notification receipt, acknowledgement, restart, first successful probe, and resolved notification. Use one time zone and correlate dashboard annotations with logs. Calculate detection/restoration times, state their start/end events, and compare observed delays with your settings.

4. Verify Jenkins HTTPS and a passing pipeline after recovery. Verify Gitea, its webhook/status integration, and the manual build. Explain how this outage differs from the failed test/build demonstrated in Chapter 4; link the earlier failed/repaired build evidence.

5. Restart Prometheus and Grafana separately without deleting volumes. Verify earlier time-series history, dashboard, data source, and alert rules remain. Record collection gaps. A restart test demonstrates persistence, not a successful backup restore.

6. Write a blameless review: impact, timeline, evidence, cause, response, measurement gaps, and two prioritized improvements recorded as Gitea issues with owners. Include the shared-droplet limitation. Merge changes through the established review and CI checks.

5.3.9 Common Problems and Checks

Grafana shows no data.
Verify the data source is `http://prometheus:9090`, the query labels match real series, the time range contains samples, and the unit is appropriate. Container `localhost` is not the droplet or another container.

A scrape target is down.
Inspect the target error and Compose logs; check service-name DNS, private port, network membership, and configuration mounts. Validate edited configuration and reload/restart the affected service.

Scrapes work but HTTPS probes fail.
Inspect the probe's HTTP status, DNS, TLS, redirects, and chosen URL. Test the route from the monitoring network. Never fix a certificate error by disabling verification.

Host values disagree.
Check Node Exporter's root/proc/sys mounts, permissions, labels, and the selected filesystem. A container overlay is not the host root disk. Compare nearby timestamps and consistent units.

No notification arrives.

Check rule state, pending period, no-data policy, contact-point credentials, notification routing, and outbound access. For email, verify SMTP configuration, sender permissions, TLS settings, and spam/junk folders. For webhooks, inspect the receiver's HTTP response and expected payload format. If a test message arrives but real alerts do not, check the rule's contact-point selection or matching notification policy, grouping delays, and silences/mute timings. If recovery messages are missing, check that resolved notifications are enabled. A dashboard threshold alone is not an alert rule.

5.4 What to Turn In: One Updated Team IT Manual PDF

Submit only the updated, compiled PDF of your existing team manual, following Chapter 1. Add a chapter titled *Introduction to SRE: Monitoring and Reliability* with the eight evidence sections below. Written operating instructions are primary; labeled screenshots, sanitized excerpts, and links support them. Do not submit separate source, screenshots, exports, or archives. Keep non-secret configuration in Gitea and link to the exact reviewed commit from the PDF.

1. **Deployment and reproducibility.** Baseline, issue/commit links, versions and publisher image references/digests, capacity/retention rationale, adapted deployment diagram showing four separate monitoring containers, startup/ update/recovery instructions, persistent paths, and restart-test results. Show Gitea, Jenkins, proxy, all monitoring tools, networks, and storage.

2. **Metrics and probes.** Sanitized scrape/probe configuration, target health, small metric inventory, host CPU/memory/root-disk verification, probe success criteria, and limitations.

3. **HTTPS and access control.** Clickable Gitea, Jenkins, and Grafana HTTPS login URLs; access instructions without passwords; proxy/firewall excerpts; authentication; and external exposure tests. A grader account is not required; embed sufficient dashboard/alert evidence for grading without private login access.

4. **Dashboard and CI correlation.** All six required panels, queries/units/ explanations, versioned dashboard-definition link, and normal-build comparison with build ID, commit, execution location, and before/during/after timestamps.

5. **SLI, SLO, and error budget.** Definitions, explained worked-example calculations, and an actual short exercise window with queries/evaluation time, sample coverage, and missing-data interpretation. Identify hypothetical figures separately from measured results; full-day SLO evaluation is optional.

6. **Alerts and runbooks.** Both final rules, owner/action instructions, no-data/error policy, notification route/delays, and timestamped firing, delivery, and resolution for each. Show the safe capacity test and restored final threshold.

7. **Incident and recovery.** Exercise plan, timeline, measured times, outage graphs, recovery/integration checks, failed-build comparison, blameless review, and two linked improvement issues with owners.

8. **Manual integration and traceability.** Updated version/history, Hosts and Passwords-hints tables, glossary, citations, and AI/reuse attribution. Identify new services, DNS names, private ports, storage, and credential hints. Link the successful automated PDF build and reviewed final commit.

5.5 Rubric (100 Points)

Rows match the eight PDF evidence sections. Credit requires implemented behavior supported by reproducible explanations and actual evidence. Installing a tool alone does not establish working queries or alerts.

#	Criterion and point allocation	Points
1	Deployment: baseline/versions/capacity/retention (4); accurate diagram (4); repeatable configuration and persistent storage (4); verified restart persistence (3).	15
2	Metrics/probes: private host/exporter/self scrapes and collection health (6); HTTPS probes for both services with defined success (5); inventory and host verification (4).	15
3	Access control: authenticated Grafana and retained Gitea/Jenkins HTTPS (4); private ports/routes, secret handling, and external exposure evidence (6).	10
4	Dashboard: six required panels with correct queries, labels, and units (8); versioned dashboard definition (3); explained normal-build comparison (4).	15
5	SLI/SLO: SLI/SLO/SLA definitions (3); explained worked-example success/budget arithmetic (3); reproducible short-window evidence, sample coverage, missing-data interpretation, and limits (4).	10
6	Alerts: two meaningful rules with owners, response steps, timing/no-data policy (6); actual firing, delivery, and resolution of both (7); safe capacity test and restored settings (2).	15
7	Incident: safe execution and correlated timeline (4); measured response and verified service/CI recovery (3); blameless review, build-failure distinction, owned improvements (3).	10
8	Manual: coherent instructions and current history/tables (4); reviewed source and successful PDF-build links (3); clear evidence, glossary, citations, attribution (3).	10

#	Criterion and point allocation	Points
	Total	**100**

Apply Chapter 1's common rules; do not deduct twice for one omission. Full-day collection and meeting the SLO are optional.

Chapter 6

Secure Delivery and Rollback

6.1 Description and Goal

Your startup can now build and test software with Jenkins and observe its Droplet with the monitoring stack. This assignment closes the delivery loop: release a small customer-facing service, verify that the intended version is running, and recover when a deployment fails. Keep the chapter title *Secure Delivery and Rollback* in your team's IT manual.

Use **Ansible** to describe and apply the desired deployment state through SSH [3]. Jenkins orchestrates tests, security gates, approvals, and Ansible runs; Ansible changes the target; Docker runs the application. **Docker Compose** is Docker's tool for declaring services, networks, and volumes in YAML [16]. You already used Compose for infrastructure services; here Ansible manages a separate Compose project for each application environment. An unchanged Ansible run should leave an already-correct deployment unchanged (*idempotence*).

This extends Chapter 4's Python test example and Chapter 5's HTTPS probes. Continue using the existing Gitea, Jenkins, reverse proxy, and monitoring containers. Do not reinstall them or add Kubernetes, Argo CD, cAdvisor, or a database for this assignment. A single Droplet is a teaching environment: staging and production-like services share a failure domain and resources. Separate Compose projects do not provide high availability or strong tenant isolation.

6.2 Starter Application and Expected Behavior

Use the supplied **Startup Pricing Service**, a stateless Flask HTTP application served by Gunicorn [52, 53]. It has no real payment processing or customer data. The starter bundle is supplied beside this manual under `Examples/dosSecureDelivery/`. Subdirectories `v1` and `v2` each provide the application, unit tests, dependency requirements, a Dockerfile, and a Docker ignore file. The parent `README.md` contains commands, prerequisites, proxy routes, deployment, and rollback instructions.

The starter uses three concepts:

- **Flask** is the Python web framework used in `app.py`. It defines the HTTP endpoints, validates requests, calculates prices, and returns responses.

- **Gunicorn** is the application server that runs the Flask code and accepts HTTP requests on port 8000 inside each application container. The supplied Dockerfile starts Gunicorn automatically when the container starts. Use it for this deployment rather than Flask's development server [52, 53].
- **Stateless** means this sample application keeps no customer records or session data that must survive a restart. Each request contains the information needed to calculate its result. Replacing an application container therefore requires no application database or persistent data volume; its selected image and configuration determine the running release.

What students install: Flask and Gunicorn are already listed in each version's `requirements.txt`. During the Docker image build, the supplied Dockerfile runs `pip install -no-cache-dir -r requirements.txt`, installing both *inside the application image*. Students build and deploy that image; they do not install Flask or Gunicorn separately on the Droplet host or in the Jenkins service container to serve the application. For unit tests run outside the application container, install the same requirements in the Python virtual environment used for those tests.

The request path is: **browser → existing HTTPS reverse proxy → Gunicorn → Flask pricing code**. Gunicorn and Flask run together inside each pricing application container; neither is a separate service container.

Request	v1	v2
`GET /`	Service name, v1, source commit	Service name, v2, source commit
`GET /health`	HTTP 200, status ok	HTTP 200, status ok
`/price?quantity=3`	Total USD 30	Total USD 36
Invalid quantity	HTTP 400	HTTP 400

The deliberate change is unit price: USD 10 becomes USD 12. A live process alone cannot prove a correct release: verify health, source identity, and the business result separately. Version and price are application source; the source commit is baked into the image at build time. Do not change an environment variable merely to make an old image identify itself as a new release. Copy v1 into your team's application directory, commit it, and later apply the v2 source and test changes in a new reviewed commit. These are two releases of one application, not two unrelated products.

6.3 Deployment Architecture

Figure 6.1 shows the required logical deployment. The build agent may share the Droplet for this lab; document its actual placement and privileges. Ansible runs on that trusted agent, not inside the application container. If that agent runs in a container, Ansible runs inside that existing agent container. The Jenkins controller schedules the job; the agent executes Ansible. SSH targets the Droplet host. Staging and production are separate containers, projects, directories, aliases, and HTTPS hostnames. The diagram builds on the earlier deployments:

retain Gitea and its chosen database, Jenkins home, proxy certificate/configuration storage, and the Prometheus and Grafana data volumes. Gray boxes distinguish durable data from read-only configuration or host mounts; not every service needs a data volume. Blackbox Exporter and Node Exporter have no data volumes, and the pricing containers are stateless. New host directories retain deployment configuration; Ansible runs on the existing agent and needs no separate service container or data volume. If PostgreSQL was selected in Chapter 2, retain that existing database service and its separate volume. Show the actual volume names and bind-mount paths in your team diagram.

Use the existing Gitea OCI container registry if enabled; a separate registry server is unnecessary [33]. Each reviewed source commit produces one image. Record its registry digest and promote that same digest to both environments. A tag helps humans find a release; a digest selects the immutable image [15]. Keep configuration commit, image digest, source commit, expected behavior, Jenkins build, approver, and timestamps together in a release record.

6.4 Implementation Steps

6.4.1 1. Prepare the Repository and Establish a Baseline

1. Create a Gitea issue and feature branch for this assignment. Add the starter application, Ansible directory, and release records to your existing team repository. Run `python -m unittest -v` in the v1 directory after installing its requirements in a virtual environment [60, 61]. The tests cover valid pricing, invalid input, and release identity. Capture a passing baseline and a controlled failing test that prevents delivery; restore the passing test.

2. The supplied dependency ranges are convenient starters, not a lockfile. Resolve and commit exact direct and transitive dependency versions before the assessed build. Select and record an approved Python base-image digest and Linux architecture compatible with the Droplet. Rebuild and rescan when you update dependencies. Never claim that a provided version is vulnerability-free.

6.4.2 2. Prepare the Trusted Agent and Target

Complete the following steps in order.

1. **Prepare the trusted Jenkins agent.** The Chapter 4 agent may not yet have container-build or deployment tools. Install Docker CLI/build capability, Python, and your chosen scanner on a *trusted* Jenkins agent; record their versions. Use a Python version supported by the selected Ansible release. When the existing agent is containerized, install these tools and Ansible inside that agent container. Installing Ansible only on the Droplet host does not make it available inside the agent container. Use the trusted agent for deployment rather than enabling deployment jobs on the Jenkins controller. A separate running Ansible container is not required.

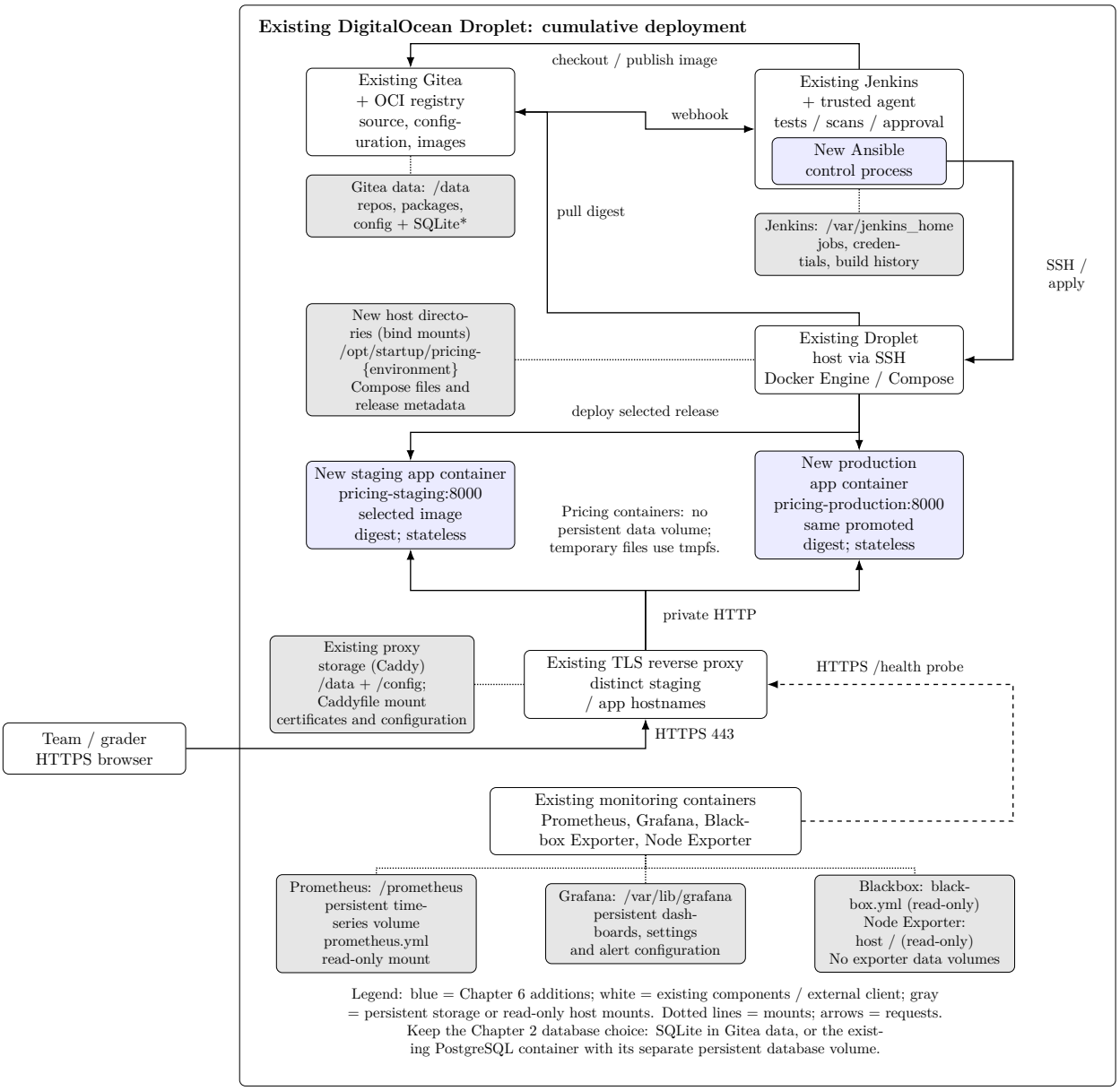


Figure 6.1: Application delivery on the existing Droplet. Arrows show the initiator of each labeled action; approval precedes production deployment. Blue identifies Chapter 6 additions, white identifies existing components, and gray identifies persistent storage or read-only host mounts. Dotted lines show mounts. Existing infrastructure and its data remain in service.

2. **Create and activate an Ansible virtual environment.** Run these commands from the checked-out repository on the trusted agent:

```
python3 -m venv .ansible-venv
. .ansible-venv/bin/activate
```

3. **Install Ansible and its Docker collection.** With that environment active, install Ansible and the supplied collection requirements, then verify and record the installed versions [4, 2]:

```
pip install ansible-core
ansible-galaxy collection install -r ansible/requirements.yml
ansible --version
ansible-galaxy collection list
```

4. **Make the agent setup reproducible.** Document the agent image or provisioning steps so that recreating its container does not lose the required tools. Activate the Ansible virtual environment in each Jenkins deployment shell step, or invoke its executables by their full paths.

5. **Verify the target Droplet prerequisites.** The target host needs Python 3, Docker Engine, and the Compose v2 plugin (at least 2.18 for this module). Verify these prerequisites before running a playbook.

6. **Configure deployment SSH access and credentials.** Create a dedicated deployment SSH account and verify the host key using a trusted channel; pin it in the agent's `known_hosts`. Store the private key and any escalation credential in Jenkins credentials [40]. The starter playbook uses `become: true`; configure approved noninteractive sudo and document its scope [5]. Docker daemon access and broad sudo are effectively host administration privileges [17]. A separate account does not remove this risk. Restrict release jobs to reviewed code and authorized users; do not expose these credentials or a Docker socket to untrusted pull-request jobs. Do not publish a Docker daemon API to the Internet [39, 48].

7. **Configure registry authentication.** Use a package-write credential on the build agent and a package-read credential on the target. Authenticate with `docker login -password-stdin`; never put tokens in Git, screenshots, command arguments, or pipeline logs. The remote Docker login must belong to the account that executes Docker under Ansible escalation (normally root), not merely the interactive SSH account [18, 33].

6.4.3 3. Connect the Application to the Existing Proxy

1. Create staging and production-like DNS names, for example `staging.example.edu` and `app.example.edu`, pointing to the Droplet. Obtain valid TLS certificates through the existing reverse proxy. Identify its Docker network; replace the `proxy_network` placeholder in each release record. The starter Compose template joins that external network with distinct aliases and publishes no host application port.

2. For a container-based Caddy proxy, add the following routes to its existing configuration, replacing the example domains. Validate and reload the proxy without removing its Gitea, Jenkins, or Grafana routes [10, 8].

```
staging.example.edu {
    reverse_proxy pricing-staging:8000
}
app.example.edu {
    reverse_proxy pricing-production:8000
}
```

3. The proxy must join the same network. If your existing proxy runs directly on the host, adapt the template to distinct loopback-only published ports and proxy to those ports. Document the adaptation. Never expose application port 8000 publicly. Preserve the firewall policy: public HTTPS, restricted SSH, and private infrastructure/metrics ports. Shared proxy-network membership is an accepted lab simplification; explain its weaker lateral isolation.

6.4.4 4. Build Once and Record an Immutable Release

1. Extend the Jenkinsfile with checkout, syntax/unit tests, dependency and secret checks, image build, image scan, and publish stages. Run Gunicorn rather than Flask's development server [52]. The starter image runs as a non-root user; its Compose template also drops capabilities and makes the root filesystem read-only with a writable temporary directory.

2. From the application directory on the trusted agent, use your registry and owner in place of the examples:

```
COMMIT=$(git rev-parse HEAD)
IMAGE=gitea.example.edu/team/pricing
docker build --build-arg APP_COMMIT="$COMMIT" -t "$IMAGE:$COMMIT" .
docker push "$IMAGE:$COMMIT"
docker pull "$IMAGE:$COMMIT"
docker image inspect "$IMAGE:$COMMIT" --format '{{index .RepoDigests 0}}'
```

3. Archive the exact repository digest, not the local image ID. Keep v1 and v2 images available throughout the exercise. Disable concurrent delivery builds so staging and production cannot be changed by competing pipeline runs [45].

6.4.5 5. Apply Proportionate Security Gates

1. Choose documented tools, for example Trivy for image vulnerabilities and repository secrets, and pip-audit for Python dependencies [7, 6, 59]. Record tool and database versions, scan time, commands, reports, and exit codes. Define a policy before running the release: block detected secrets and high/critical vulnerabilities unless the instructor accepts a

documented, time-limited exception. State how fixes, unfixed issues, false positives, and exceptions are handled. Do not silently ignore findings or rely only on a green badge.

2. Demonstrate one blocked pipeline using a scanner-supported synthetic test fixture. Never commit a live secret or publish a deliberately vulnerable image as the working service. Remove the fixture, fix or triage the result, rerun, and retain redacted evidence. Fail the pipeline when a scanner errors or its database cannot be obtained; a failed scan is not a clean scan.

6.4.6 6. Deploy v1 with Ansible and Prove Idempotence

1. The supplied `Examples/dosSecureDelivery/ansible/` directory contains `inventory.ini`, `deploy.yml`, a Compose Jinja template, and `release.example.yml`. Replace every placeholder. Copy the example release file to `releases/v1-staging.yml`; set the image digest, full source commit, expected version v1, expected total 30, staging URL, and proxy network. These release records contain metadata, never credentials.

2. ```
   ansible-playbook -i ansible/inventory.ini ansible/deploy.yml --syntax-check
   ansible-playbook -i ansible/inventory.ini ansible/deploy.yml \
     -e @releases/v1-staging.yml
   ```

   The playbook validates inputs, creates an environment directory, renders Compose, applies the selected image without rebuilding, waits for container health, and checks the public HTTPS health, version, commit, and price. See the Compose v2 and HTTP module documentation for these operations [2, 1]. Application state lives under `/opt/startup/pricing-staging` or `/opt/startup/pricing-production`; no database volume is needed.

3. Run exactly the same deployment a second time and show no unnecessary changes (normally `changed=0`). Investigate plugin/version differences or changing templates if this is not true. A syntax check alone does not prove successful deployment. The supplied playbook fails on a failed health or behavior check; it does not silently perform an automatic rollback.

6.4.7 7. Approve and Promote the Same Image

1. Add Jenkins stages for staging deployment and smoke tests, a human approval, and production deployment and smoke tests. Use Jenkins' `input` step with an authorized submitter restriction and a timeout. Present the source commit, image digest, scan result, and staging result to the approver. Record the actual approver and decision [44, 45]. An unattended approval or a prompt that any anonymous user can approve does not satisfy this requirement.

2. Create `v1-production.yml` with the same digest and source commit but the production environment and URL. Promote without rebuilding. Stop when approval is rejected or times out. Archive redacted release records and reports even if a later deployment
   ```

fails. Your team implements this Jenkinsfile integration; the starter directory deliberately provides the application and Ansible files, not a completed assessed pipeline.

6.4.8 8. Release v2 and Exercise Two Recovery Cases

1. Commit the supplied v2 application and tests, run the full pipeline, and verify version v2, its exact source commit, and a total of 36 in both approved environments. Preserve the v1 release and configuration records.

2. Carry out both recovery exercises:

   (a) **Healthy release rollback:** redeploy the reviewed v1 image and its matching configuration to the production-like environment using Ansible. Verify v1, its source commit, health, and total 30. Record detection/decision, rollback start, recovery time, and observed interruption.

   (b) **Failed deployment recovery in staging only:** in a separate configuration commit, override the container command to bind Gunicorn to 8001 while the image healthcheck still tests 8000. The README provides the exact command. Show that Compose wait fails and the Jenkins run stops. Restore the previous reviewed template/configuration commit and its release record; rerun Ansible and verify HTTPS health and business behavior. Do not alter the host Docker daemon, proxy, Gitea, or Jenkins to create the failure.

3. Rollback includes the *configuration commit as well as the image digest*. Do not delete shared volumes or use `docker compose down -v` or system pruning. This recreate deployment can briefly interrupt traffic; measure it rather than claiming zero downtime. The example has no database to restore. Explain why a real database migration, external payment, or incompatible stored state could prevent safe image-only rollback, and describe one compatible migration approach without implementing a database.

6.4.9 9. Reuse Monitoring and Document Operations

1. Add the production application's HTTPS `/health` endpoint to Chapter 5's Blackbox Exporter probe targets and display its availability in Grafana [54, 36]. Keep the existing probes for Gitea and Jenkins, together with host metrics [57, 56]. Correlate the release and rollback timestamps with monitoring evidence. A short failure can occur between scrapes or below the alert duration: report what was actually observed, not an assumed alert. Keep version and pricing smoke tests in the pipeline; a status-only uptime probe cannot validate the price.

2. Write release, failed-deployment, rollback, and cleanup runbooks. Identify who can approve, deploy, and roll back. Record resource use and the single-Droplet limitation. After grading, remove only the application projects, their proxy routes, test DNS records and credentials when instructed; preserve the earlier assignments and any evidence needed for assessment.

6.5 What to Turn In

Submit your updated **team IT manual as one PDF**, following the course submission convention. Keep application source and automation in Gitea and include clickable links to reviewed commits and Jenkins runs, with grader access instructions. Do not submit credentials or require a running pipeline to understand the report. Include:

1. Architecture diagram showing actual hosts, container boundaries, Gitea/registry, Jenkins/agent, Ansible SSH, both environments, proxy, and monitoring. Explain public versus private connections and deployment rights.

2. Installation/version inventory and exact dependency/base-image selection; links to both application releases, tests, Dockerfile, Jenkinsfile, inventory with secrets removed, playbook, template, and release records.

3. A traceability table for v1 and v2: source/configuration commits, image digests, build IDs, scan results, staging results, approver/time, production results, and corresponding evidence links.

4. Redacted evidence of passing and blocked gates, authenticated registry publication/pull, Ansible deployment and unchanged second run, HTTPS health/version/commit/price tests, v2 promotion, and both recovery cases.

5. Timestamped rollback timeline, observed monitoring results and interruption, discussion of limitations and state compatibility, runbooks, cleanup plan, updated document history, and AI-assistance log where applicable.

Screenshots support the explanation; label them with environment, release, and time. A tool installation screenshot without a demonstrated release and recovery is insufficient.

6.6 Rubric (100 Points)

Criterion and evidence	**Points**
Working v1/v2 service, meaningful tests, source identity and locked build inputs	10
Immutable image publication and traceability; same digest promoted	10
Ansible deployment, environment separation, valid HTTPS and unchanged rerun	20
Jenkins gates, restricted recorded approval and verified v2 promotion	15
Security scan policy, reports, blocked fixture and credential/access controls	15
Both recovery cases, matching configuration restoration and measured verification	15
Monitoring correlation and clear operational/state limitations	5
Complete PDF manual, diagram, release records, accessible links and runbooks	10
Total	**100**

6.7 Official References

The numbered citations above link to the manual's bibliography, which includes clickable documentation URLs. Use documentation matching your installed versions. The starter application and deployment diagram are course examples; the references describe the underlying tools and their supported behavior.

- **Ansible:** control and managed nodes, installation, collection requirements, Compose deployment, privilege escalation, and HTTPS checks [3, 4, 2, 5, 1].
- **Docker and Gitea:** Compose projects, reproducible image builds, registry authentication, publication, and daemon security [16, 15, 18, 33, 17].
- **Jenkins:** Pipeline stages and concurrency, restricted human approval, credential storage, and trusted execution boundaries [45, 44, 40, 39, 48].
- **Sample service:** Flask production deployment with Gunicorn, Python virtual environments, and unit tests [52, 53, 61, 60].
- **Security gates:** Trivy vulnerability and secret scanning, and PyPA pip-audit dependency auditing [7, 6, 59].
- **Proxy and monitoring:** Caddy reverse proxy and automatic HTTPS; Prometheus configuration, Blackbox Exporter, Node Exporter, and Grafana's Prometheus data source [10, 8, 57, 54, 56, 36].

Appendix A

Student IT Manual Update Guide

The assignment manual tells teams what to do; the team's own DevOps IT manual records what they actually built, tested, and learned. Start from `main/DevOpsTemplate/` and preserve its versioning, history, glossary, index, citation, and consistent chapter conventions. Add one operational chapter per capability as it is delivered. A new chapter should explain purpose, architecture, configuration, verification, security, failure modes, recovery, ownership, and cost/cleanup. Link figures, commands, repository files, and evidence with stable labels. Do not copy an assignment brief as if it were an implementation report.

A.1 Minimum Table Fields

Table	**Suggested fields**
Hosts	Hostname, role, environment, provider, public/private address or DNS, exposed ports, owner, HTTPS URL, backup/monitoring status.
Passwords	Service/account, owner, approved vault reference or non-revealing hint, privilege level, MFA/rotation status. Never the secret itself.
History	Date, version, author, what changed, why, and linked chapter/table or test evidence. Mention Hosts and Passwords updates explicitly.

A.2 Evidence Standard

Separate *planned*, *configured*, and *verified* work. State the test date, expected result, actual result, and evidence location. Include negative results and fixes. Screenshots should be legible and captioned, but versioned configuration and commands are usually more reproducible. Before submission, verify hyperlinks, citations, compilation, redaction, and the consistency of every table and architecture diagram.

Appendix B

AI Agent Usage Guide

This LaTeX chapter is the authoritative AI-maintenance directive for the assignment manual. An AI agent should read it, the introduction, the affected assignment chapter, and the current history before editing. Make changes only to the requested scope. The instructor-approved file namespace is `dos`. Project-owned filenames begin with `dos`, followed by a PascalCase responsibility and file extension (for example, `dosStartupFoundation.tex`). The root source is `dosManual.tex`. `prologue.tex`, `cornell.cls`, and `Makefile` are retained exceptions for front matter, an externally maintained class, and a conventional build entry point; generated build files are not authored sources.

Keep one responsibility per chapter. A new assignment chapter needs a description, observable deliverables, explicit student-manual PDF evidence, and a rubric totaling exactly 100 points. Keep future assignment chapters commented out until the instructor activates them. Place each TikZ source at `TikZ/<including-file-base>/<dosDrawingName>.tex`; the directory must exactly match the base name of the chapter containing its `\input`. The TikZ file contains only the `tikzpicture`; the owning chapter contains the figure environment, caption, label, and citations. Keep substantive AI instructions in this LaTeX chapter and log material AI-assisted work in Appendix C; any tool-discovery file should point here rather than duplicate these rules.

Do not invent due dates, budgets, eligibility, tested outcomes, security posture, cloud access, grades, or instructor approval. Treat the LaTeX source as authoritative and the PDF as a review artifact. Never include secrets, tokens, private keys, student personal data, or unredacted production logs. Verify current external product documentation before adding instructions that depend on versions or offers. Update the document version and history for content changes. Build the complete PDF, inspect the log and affected pages, and record material AI assistance in Appendix C. The instructor decides assignment scope, deadlines, exceptions, and grading policy.

Appendix C

AI Assistance Log

Record material AI-assisted edits so later instructors can distinguish human-approved requirements from generated draft text. Do not paste private student data or credentials into prompts or this log.

Date	Tool / Reviewer	Task and human verification
2026-10-28	OpenAI Codex / D. Muresan	Expanded Chapter 6 at instructor request with Ansible/Compose delivery, a deployment diagram, starter pricing releases, security and approval gates, recovery exercises, evidence and rubric. Official tool documentation consulted. Local application and build checks recorded in the working session; no student infrastructure deployment was performed. Instructor review of workload and release policy remains pending.
2026-10-21	OpenAI Codex / D. Muresan	Simplified Chapter 5 after instructor feedback: excluded cAdvisor and per-container attribution, retained four project-image containers, made application metrics and full-day SLO collection optional, supplied the HTTPS probe job, and aligned six panels, evidence, rubric, and deployment diagram. Added contact-point creation, email SMTP prerequisites, webhook/integration selection, rule routing, delivery tests, and troubleshooting guidance. Configuration guidance was checked against official documentation; no student droplet was deployed. Updated the manual and checked compilation and PDF layout.

2026-10-14	OpenAI Codex / D. Muresan	Drafted the requested introductory SRE assignment for the existing Gitea/Jenkins droplet, with Prometheus, Grafana, exporters, deployment diagram, implementation steps, submission evidence, and rubric. Official documentation was consulted; no student infrastructure was deployed or tested. Widened Figure 5.1 and adjusted its labels and routes. Added four separate project-image containers, a Compose installation scaffold, host-metric validation, and orthogonal diagram connectors; container deployment itself was not executed. The project PDF compiled and the new chapter was checked visually; instructor review of workload and grading remains pending.
2026-10-07	OpenAI Codex / D. Muresan	Revised the Jenkins continuous-integration assignment: clarified the protected-branch merge gate, added an activity diagram, a small test application, and an optional sample Jenkinsfile, and refined the PDF evidence requirements and rubric. Added a Gitea issue-linked pull-request example and adjusted deployment-diagram spacing. The project PDF compiled; the instructor should review the technical requirements, examples, and grading policy before release.
2026-09-30	OpenAI Codex / D. Muresan	Drafted the Gitea Actions assignment for building the LaTeX manual PDF, including workflow and deployment diagrams, runner/toolchain guidance, and HTTPS Git submission paths. The project PDF compiled; the instructor should verify the intended student workload, deployment choices, and rubric before release.
2026-09-24	OpenAI Codex / D. Muresan	Applied the instructor-approved `dos` namespace, aligned the TikZ source layout and AI chapters with the architecture template, and added a Chapter 2 Gitea deployment reference. Built and checked the PDF; the instructor should confirm the diagram fits the intended student deployment choices.

2026-09-23	OpenAI Codex / D. Muresan	Drafted modular startup DevSecOps assignment chapters and rubrics from the instructor's stated course goals; the instructor should review scope, order, deadlines, and rubric policy before release. LaTeX build/visual checks recorded in the working session.

Bibliography

[1] Ansible Community. ansible.builtin.uri: Interacts with Webservices. https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/uri_module.html. Accessed 2026-10-28.

[2] Ansible Community. community.docker.docker_compose_v2: Manage Docker Compose Applications. https://docs.ansible.com/projects/ansible/latest/collections/community/docker/docker_compose_v2_module.html. Accessed 2026-10-28.

[3] Ansible Community. Getting Started with Ansible. https://docs.ansible.com/projects/ansible/latest/getting_started/index.html. Accessed 2026-10-28.

[4] Ansible Community. Installing Ansible. https://docs.ansible.com/projects/ansible/latest/installation_guide/intro_installation.html. Accessed 2026-10-28.

[5] Ansible Community. Understanding Privilege Escalation: become. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_privilege_escalation.html. Accessed 2026-10-28.

[6] Aqua Security. Trivy: Secret Scanning. https://trivy.dev/docs/latest/scanner/secret/. Accessed 2026-10-28.

[7] Aqua Security. Trivy: Vulnerability Scanning. https://trivy.dev/docs/latest/scanner/vulnerability/. Accessed 2026-10-28.

[8] Caddy. Automatic https. https://caddyserver.com/docs/automatic-https. Accessed 2026-09-24.

[9] Caddy. Keep caddy running: Docker compose. https://caddyserver.com/docs/running#docker-compose. Accessed 2026-09-24.

[10] Caddy. Reverse proxy quick-start. https://caddyserver.com/docs/quick-starts/reverse-proxy. Accessed 2026-09-24.

[11] CTAN. Latexmk: Fully automated latex document generation. https://ctan.org/pkg/latexmk. Accessed 2026-09-30.

[12] DigitalOcean. Firewalls quickstart. https://docs.digitalocean.com/products/networking/firewalls/getting-started/quickstart/. Accessed 2026-09-23.

[13] DigitalOcean. How to configure firewall rules. https://docs.digitalocean.com/products/networking/firewalls/how-to/configure-rules/. Accessed 2026-09-23.

[14] Docker. Packet filtering and firewalls. https://docs.docker.com/engine/network/packet-filtering-firewalls/. Accessed 2026-09-23.

[15] Docker Inc. Building Best Practices. https://docs.docker.com/build/building/best-practices/. Accessed 2026-10-28.

[16] Docker Inc. Docker Compose. https://docs.docker.com/compose/. Accessed 2026-10-28.

[17] Docker Inc. Docker Engine Security. https://docs.docker.com/engine/security/. Accessed 2026-10-28.

[18] Docker Inc. docker login: Authenticate to a Registry. https://docs.docker.com/reference/cli/docker/login/. Accessed 2026-10-28.

[19] Gitea. Automatically linked references. https://docs.gitea.com/usage/issues-prs/automatically-linked-references/. Accessed 2026-10-07.

[20] Gitea. Configuration cheat sheet. https://docs.gitea.com/administration/config-cheat-sheet/. Accessed 2026-09-24.

[21] Gitea. Configuration cheat sheet: Metrics. https://docs.gitea.com/administration/config-cheat-sheet/#metrics-metrics. Accessed 2026-10-14.

[22] Gitea. Database preparation. https://docs.gitea.com/installation/database-prep/. Accessed 2026-09-23.

[23] Gitea. Gitea actions artifacts. https://docs.gitea.com/usage/actions/artifacts/. Accessed 2026-09-30.

[24] Gitea. Gitea actions quick start. https://docs.gitea.com/usage/actions/quickstart/. Accessed 2026-09-30.

[25] Gitea. Gitea actions variables. https://docs.gitea.com/usage/actions/actions-variables/. Accessed 2026-09-30.

[26] Gitea. Gitea runner labels. https://docs.gitea.com/runner/labels/. Accessed 2026-09-30.

[27] Gitea. Install gitea runner with docker. https://docs.gitea.com/runner/2/installation/docker/. Accessed 2026-09-30.

[28] Gitea. Installation with docker. https://docs.gitea.com/installation/install-with-docker/. Accessed 2026-09-23.

[29] Gitea. Protected branches. https://docs.gitea.com/usage/access-control/protected-branches/. Accessed 2026-09-23.

[30] Gitea. Pull requests. https://docs.gitea.com/usage/issues-prs/pull-request/. Accessed 2026-10-07.

[31] Gitea. Reverse proxies. https://docs.gitea.com/administration/reverse-proxies/. Accessed 2026-09-30.

[32] Gitea. Webhooks. https://docs.gitea.com/usage/repository/webhooks/. Accessed 2026-09-23.

[33] Gitea Authors. Container Registry. https://docs.gitea.com/usage/packages/container/. Accessed 2026-10-28.

[34] GitHub Education. Github student developer pack: Domain offers. https://education.github.com/pack?sort=az&tag=Domains. Offer eligibility and terms should be checked at the time of use.

[35] Google. Implementing slos: The site reliability workbook. https://sre.google/workbook/implementing-slos/. Accessed 2026-10-14.

[36] Grafana Labs. Configure the prometheus data source. https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/. Accessed 2026-10-14.

[37] Grafana Labs. Grafana alerting. https://grafana.com/docs/grafana/latest/alerting/. Accessed 2026-10-14.

[38] Grafana Labs. Run grafana docker image. https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/. Accessed 2026-10-14.

[39] Jenkins Project. Controller isolation. https://www.jenkins.io/doc/book/security/controller-isolation/. Accessed 2026-09-23.

[40] Jenkins Project. Credentials. https://www.jenkins.io/doc/book/security/credentials/. Accessed 2026-09-23.

[41] Jenkins Project. Gitea checks plugin. https://plugins.jenkins.io/gitea-checks/. Accessed 2026-09-23.

[42] Jenkins Project. Gitea plugin. https://plugins.jenkins.io/gitea/. Accessed 2026-09-23.

[43] Jenkins Project. Installing jenkins with docker. https://www.jenkins.io/doc/book/installing/docker/. Accessed 2026-09-23.

[44] Jenkins Project. Pipeline: Input Step. https://www.jenkins.io/doc/pipeline/steps/pipeline-input-step/. Accessed 2026-10-28.

[45] Jenkins Project. Pipeline syntax. https://www.jenkins.io/doc/book/pipeline/syntax/. Accessed 2026-10-07.

[46] Jenkins Project. Prometheus metrics plugin. https://plugins.jenkins.io/prometheus/. Accessed 2026-10-14.

[47] Jenkins Project. Reverse proxy configuration. https://www.jenkins.io/doc/book/system-administration/reverse-proxy-configuration-with-jenkins/. Accessed 2026-09-23.

[48] Jenkins Project. Securing jenkins. https://www.jenkins.io/doc/book/security/. Accessed 2026-09-23.

[49] Jenkins Project. Using a jenkinsfile. https://www.jenkins.io/doc/book/pipeline/jenkinsfile/. Accessed 2026-10-07.

[50] Let's Encrypt. Challenge types. https://letsencrypt.org/docs/challenge-types/. Accessed 2026-09-23.

[51] Let's Encrypt. Let's encrypt: Free tls certificates. https://letsencrypt.org/. Accessed 2026-09-23.

[52] Pallets Projects. Flask: Deploying to Production. https://flask.palletsprojects.com/en/stable/deploying/. Accessed 2026-10-28.

[53] Pallets Projects. Flask: Gunicorn. https://flask.palletsprojects.com/en/stable/deploying/gunicorn/. Accessed 2026-10-28.

[54] Prometheus Authors. Blackbox exporter. https://github.com/prometheus/blackbox_exporter. Accessed 2026-10-14.

[55] Prometheus Authors. Installation and docker persistence. https://prometheus.io/docs/prometheus/latest/installation/. Accessed 2026-10-14.

[56] Prometheus Authors. Node exporter: Host and container deployment. https://github.com/prometheus/node_exporter. Accessed 2026-10-14.

[57] Prometheus Authors. Prometheus configuration. https://prometheus.io/docs/prometheus/latest/configuration/configuration/. Accessed 2026-10-14.

[58] Prometheus Authors. Prometheus query functions. https://prometheus.io/docs/prometheus/latest/querying/functions/. Accessed 2026-10-14.

[59] Python Packaging Authority. pip-audit: Audit Python Dependencies for Known Vulnerabilities. https://github.com/pypa/pip-audit. Accessed 2026-10-28.

[60] Python Software Foundation. unittest: Unit Testing Framework. https://docs.python.org/3/library/unittest.html. Accessed 2026-10-28.

[61] Python Software Foundation. venv: Creation of Virtual Environments. https://docs.python.org/3/library/venv.html. Accessed 2026-10-28.

[62] TeX Users Group. Tex live. https://tug.org/texlive/. Accessed 2026-09-30.

Glossary

DevSecOps A DevOps practice that integrates security throughout planning, building, delivery, and operations. 59

error budget The amount of unreliability permitted by a service-level objective over its measurement window. 59

evidence A retained, sanitized record of an actual execution, observation, or review. 59

exporter A component that exposes measurements in a format Prometheus can scrape. 59

least privilege Giving an account or service only the access it needs for its assigned task. 59

reverse proxy A service that receives external requests and forwards them to internal services; it may terminate HTTPS. 59

rollback A controlled return to a known acceptable version or state. 59

runbook A documented, repeatable operational procedure. 59

Acronyms

AI artificial intelligence. 59

CD continuous delivery. 59

CI continuous integration. 59

MFA multi-factor authentication. 59

SLA service-level agreement. 59

SLI service-level indicator. 23, 59

SLO service-level objective. 23, 59

SRE site reliability engineering. 23, 59

TLS Transport Layer Security. 59

Index

AI agent, 51
AI assistance log, 52
Ansible, 40

build artifact, 9

continuous integration, 15

database, 5
Docker Compose, 40
document history, iv

evidence, 2

firewall, 6

Git push, 10
Gitea, 3
Gitea Actions, 9
Grafana, 23

HTTPS, 2, 10

incident response, 23
IT manual, 50

Jenkins, 15
job container, 10

LaTeX automation, 9
latexmk, 10

monitoring, 23

Prometheus, 23

reverse proxy, 10
rollback, 40

secure delivery, 40
site reliability engineering, 23
startup DevSecOps process, 1

TeX Live, 10